Rent my car

Studiegids bij de code van de Ktor Web API
Alles uitgelegd, bestand voor bestand

Avans – module AI-powered software development, leeruitkomst 1
Stand van de code: branch `feature/domain-classes`, 22 oktober 2026
Gemaakt met hulp van Claude Code (AI), als studiemateriaal voor het assessment

Inhoud

1. Hoe gebruik je dit document?
2. Het grote plaatje
3. Opbouw van de code: mappen, bestanden en namen
4. Gebruikte technieken en libraries
5. Begrippen van een Web API (HTTP, REST, JSON)
6. Opstarten: van `main()` tot draaiende server
7. De module `core`: DTO's en enums
8. Het domein: User, Car en Reservation
9. Opslag: tabellen en repositories (Exposed + H2)
10. Services: de use cases en de regels
11. Routes: de HTTP-kant (Ktor)
12. Foutafhandeling: van exception naar statuscode
13. Beveiliging
14. Testen
15. Eén request van begin tot eind
16. Kotlin-begrippen uitgebreid uitgelegd
17. Ontwerppatronen en hoe je ze noemt
18. Spiekbrief Kotlin-concepten
19. Oefenvragen voor het assessment
20. Aandachtspunten in de huidige code

1. Hoe gebruik je dit document?

Dit document legt de code van de server uit, in dezelfde volgorde als waarin een verzoek (request) door de applicatie gaat. Bij elk onderdeel staat:

- **wat** het bestand doet en waarom het bestaat;
- een **stukje code** met uitleg;
- de **Kotlin-concepten** die je moet kunnen uitleggen;
- waar nuttig: **waarom** er voor deze oplossing gekozen is.

> **Studietip**
> Open bij het lezen het bestand in IntelliJ ernaast. Probeer na elk hoofdstuk zonder te kijken uit te leggen wat een bestand doet. Het hoofdstuk Oefenvragen heeft vragen zoals ze bij een assessment gesteld kunnen worden.

> **Over de herkomst**
> Dit document beschrijft de code zoals die nu in de repository staat. De uitleg is door de AI geschreven op basis van die code. Waar de code zelf zegt waar een keuze vandaan komt (bijvoorbeeld de Argon2-instellingen "zoals in de demo van de gastdocent"), staat dat erbij. Controleer de uitleg zelf: jij moet het bij het assessment kunnen verdedigen.

1.1 Woordenlijst

Term	Betekenis
API	Een programma dat geen schermen heeft, maar verzoeken via HTTP beantwoordt met data (hier JSON).
Endpoint / route	Een combinatie van HTTP-methode en pad, bijvoorbeeld `POST /api/cars`.
JSON	Tekstformaat voor data: `{ "brand": "Tesla", "seats": 5 }`.
DTO	Data Transfer Object: een klasse die alleen bedoeld is om data van/naar de client te sturen.
Domein	De klassen die de "echte wereld" beschrijven (auto, gebruiker, reservering) met hun regels.
Repository	Een klasse die objecten opslaat en terugvindt, zodat de rest van de code de database niet hoeft te kennen.
Service	Een klasse met de use cases: "auto toevoegen", "reservering goedkeuren". Hier zitten de controles.
JWT	JSON Web Token: een ondertekend stukje tekst dat bewijst wie er is ingelogd.
Hash	Een onomkeerbare "vingerafdruk" van een wachtwoord. Je kunt er het wachtwoord niet uit terugrekenen.
Statuscode	Getal in een HTTP-antwoord: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 429 Too Many Requests, 500 Server Error.

2. Het grote plaatje

2.1 Twee modules: `core` en `server`

Het project gebruikt de Kotlin Toolchain (`kotlin.bat`, geen Gradle). Elke module heeft een `module.yaml`.

Module	Soort	Wat zit erin
`core`	`kmp/lib` (Kotlin Multiplatform-bibliotheek, voor JVM en Android)	De DTO's en enums die de server én (in periode 2) de Android-app allebei nodig hebben. Package `rentmycar.core`.
`server`	`jvm/app`	De Ktor-server. Hangt af van `//core` en van Ktor, Exposed, H2, BouncyCastle en Logback. Package `rentmycar.server.*`.

Waarom een aparte `core`? Zo gebruiken server en app precies dezelfde klassen voor de JSON. Als je een veld toevoegt aan `CarResponse`, zien beide kanten dat meteen.

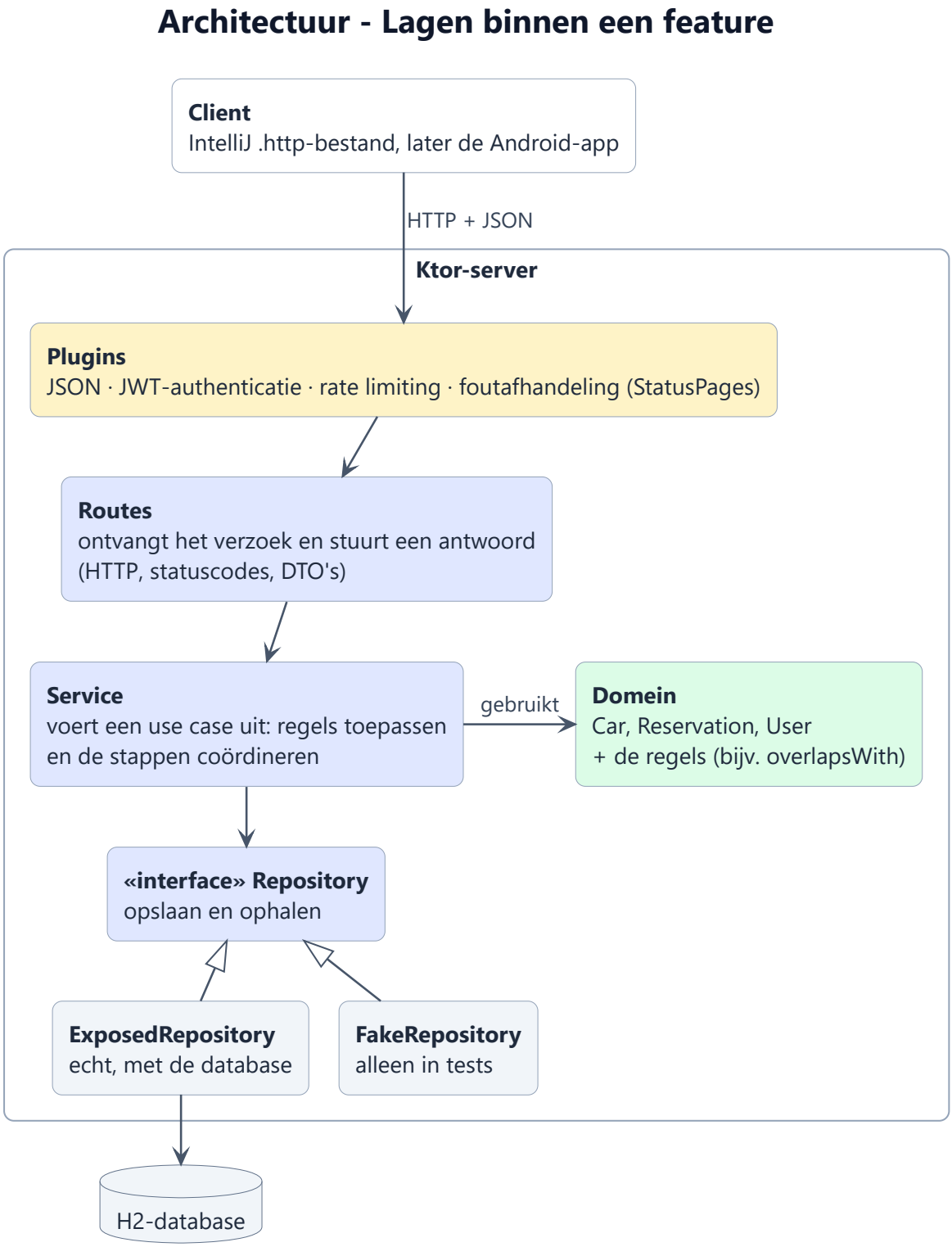
De versies van de libraries staan centraal in `libs.versions.toml` (bijvoorbeeld Exposed 1.5.0, H2 2.4.240, BouncyCastle 1.84). In `module.yaml` verwijs je ernaar met `$libs.exposed.core`.

2.2 De lagen binnen een feature

De server is verdeeld in features (packages): `users`, `cars`, `reservations` en het gedeelde `common`. Binnen elke feature zijn dezelfde lagen:

```
HTTP-request
   │
   ▼
Routes      (XxxRoutes.kt)       ← leest JSON/URL, roept de service aan, stuurt antwoord
   │
   ▼
Service     (XxxService.kt)      ← use cases, eigenaarschap (403), dubbele dingen (409)
   │
   ├──► Domein (Car, Reservation, User)   ← regels die altijd gelden (require/check)
   │
   ▼
Repository  (interface + ExposedXxxRepository)  ← opslaan en ophalen
   │
   ▼
Table       (XxxTable.kt)  → H2-database
```

Daarnaast heeft elke feature een **Mapping**-bestand (`toResponse()`) dat een domeinobject omzet naar een DTO.



Figuur 1 – Architectuur uit docs/images/architecture.svg

2.3 Waarom deze lagen?

- **Elke laag één taak**: routes weten niets van SQL, de database weet niets van HTTP.
- **Testbaar**: omdat de service een *interface* `CarRepository` krijgt, kun je in een test een nep-versie (fake) meegeven zonder database (zie Testen).
- **Regels op één plek**: dat een uurtarief niet negatief mag zijn staat in `Car`, niet verspreid over routes.

2.4 Overzicht van alle bestanden

Bestand	Taak in één zin
`Server.kt`	Bevat `main()`: leest het JWT-geheim, maakt de onderdelen, vult demodata, start Netty op poort 8080.
`Dependencies.kt`	Maakt alle repositories en services één keer en koppelt ze aan elkaar.
`Application.kt`	Installeert de Ktor-plugins (JSON, StatusPages, RateLimit, Authentication) en de routes.
`DatabaseFactory.kt`	Verbindt met H2 (bestand of geheugen) en maakt de tabellen.
`DemoData.kt`	Vult een lege database met 3 gebruikers, 7 auto's en 2 reserveringen.
`common/*`	Gedeeld: kostenaannames, exceptions, StatusPages, afronden, tijd, hulpfuncties voor de call.
`users/*`	Registreren, inloggen, wachtwoord-hash (Argon2id), JWT-tokens.
`cars/*`	Auto's (3 soorten), zoeken, toevoegen, wijzigen, verwijderen, kosten (TCO).
`reservations/*`	Reservering aanvragen, bekijken, goedkeuren/afwijzen/annuleren.

3. Opbouw van de code: mappen, bestanden en namen

3.1 De mappenstructuur

```
RentMyCar_Periode1/
├── kotlin.bat                ← start de Kotlin Toolchain (bouwen, draaien, testen)
├── libs.versions.toml        ← versies van alle libraries op één plek ("version catalog")
├── core/                     ← gedeelde module (server + later de Android-app)
│   ├── module.yaml           ← soort module + afhankelijkheden
│   └── src/
│       ├── ApiError.kt       ← body van elk foutantwoord
│       ├── CarType.kt        ← enum ICE / BEV / FCEV
│       ├── FuelType.kt       ← enum PETROL / DIESEL / LPG
│       ├── ReservationStatus.kt
│       ├── UserDtos.kt       ← JSON-klassen voor gebruikers
│       ├── CarDtos.kt        ← JSON-klassen voor auto's
│       └── ReservationDtos.kt ← JSON-klassen voor reserveringen
├── server/
│   ├── module.yaml
│   ├── resources/logback.xml  ← instellingen voor de log-uitvoer
│   ├── src/
│   │   ├── Server.kt          ← main(): startpunt van het programma
│   │   ├── Application.kt     ← Ktor-plugins + routes
│   │   ├── Dependencies.kt    ← alles aan elkaar koppelen
│   │   ├── DatabaseFactory.kt ← verbinding met H2 + tabellen maken
│   │   ├── DemoData.kt        ← voorbeelddata bij een lege database
│   │   ├── common/            ← gedeeld door alle features
│   │   │   ├── CallUtils.kt       (userId(), intParameter())
│   │   │   ├── CostAssumptions.kt (vaste getallen kostenberekening)
│   │   │   ├── Exceptions.kt      (ForbiddenException, ConflictException)
│   │   │   ├── Rounding.kt        (Double.rounded())
│   │   │   ├── StatusPages.kt     (exception → statuscode)
│   │   │   └── TimeUtils.kt       (currentTime(), hoursBetween())
│   │   ├── users/             ← feature "gebruikers"
│   │   │   ├── User.kt            (domein)
│   │   │   ├── UserTable.kt       (databasetabel)
│   │   │   ├── UserRepository.kt  (interface + Exposed-versie)
│   │   │   ├── UserService.kt     (use cases)
│   │   │   ├── UserRoutes.kt      (HTTP)
│   │   │   ├── UserMapping.kt     (domein → DTO)
│   │   │   ├── PasswordHasher.kt  (Argon2id)
│   │   │   └── TokenService.kt    (JWT)
│   │   ├── cars/              ← feature "auto's" (zelfde opbouw)
│   │   │   ├── Car.kt, IceCar.kt, ElectricCar.kt, HydrogenCar.kt, Address.kt
│   │   │   ├── CarTable.kt, CarRepository.kt, CarService.kt
│   │   │   └── CarRoutes.kt, CarMapping.kt
│   │   └── reservations/      ← feature "reserveringen" (zelfde opbouw)
│   └── test/
│       ├── TestData.kt, Fakes.kt
│       ├── CarCostsTest.kt, ReservationTest.kt, ServiceTest.kt   (unit)
│       ├── CarRepositoryTest.kt, ApiTest.kt                     (integratie)
│       └── http-requests/*.http                                 (handmatig)
└── docs/                     ← ontwerp, diagrammen (puml → images/*.svg), AI-log
```

3.2 Indeling per feature ("package by feature")

De code is niet ingedeeld per *soort* bestand (een map met alle routes, een map met alle services), maar per **feature**: alles van auto's staat in `cars/`. Dit heet **package by feature**. Voordeel: als je iets aan auto's

verandert, zit alles bij elkaar. Binnen elke feature is de bestandsindeling steeds hetzelfde, dus als je één feature snapt, snap je ze alle drie.

Achtervoegsel	Rol (laag)	Voorbeeld
(geen)	Domeinklasse: het "ding" zelf met zijn regels	`Car`, `Reservation`, `User`
`...Table`	Beschrijving van een databasetabel (Exposed)	`CarTable`
`...Repository`	Opslaan en ophalen	`CarRepository`, `ExposedCarRepository`
`...Service`	Use cases en controles	`CarService`
`...Routes`	HTTP-endpoints	`carRoutes()`
`...Mapping`	Omzetten domein → DTO	`Car.toResponse()`
`...Request` / `...Response` / `...Dto`	JSON die binnenkomt / uitgaat	`CarRequest`, `CarResponse`, `AddressDto`
`Fake...`	Nepversie voor tests	`FakeCarRepository`

3.3 Packages

Elk bestand begint met een `package`-regel, bijvoorbeeld `package rentmycar.server.cars`. Een package is een "achternaam" voor klassen: zo kunnen twee klassen met dezelfde naam in verschillende packages bestaan, en zie je meteen waar iets bij hoort. Om iets uit een ander package te gebruiken schrijf je een `import`, bijvoorbeeld `import rentmycar.core.CarType`.

Package	Inhoud
`rentmycar.core`	DTO's en enums (module core)
`rentmycar.server`	Opstarten en koppelen (Server, Application, Dependencies, DatabaseFactory, DemoData)
`rentmycar.server.common`	Gedeelde hulpcode
`rentmycar.server.users` / `.cars` / `.reservations`	De drie features

3.4 Naamgeving (naming conventions)

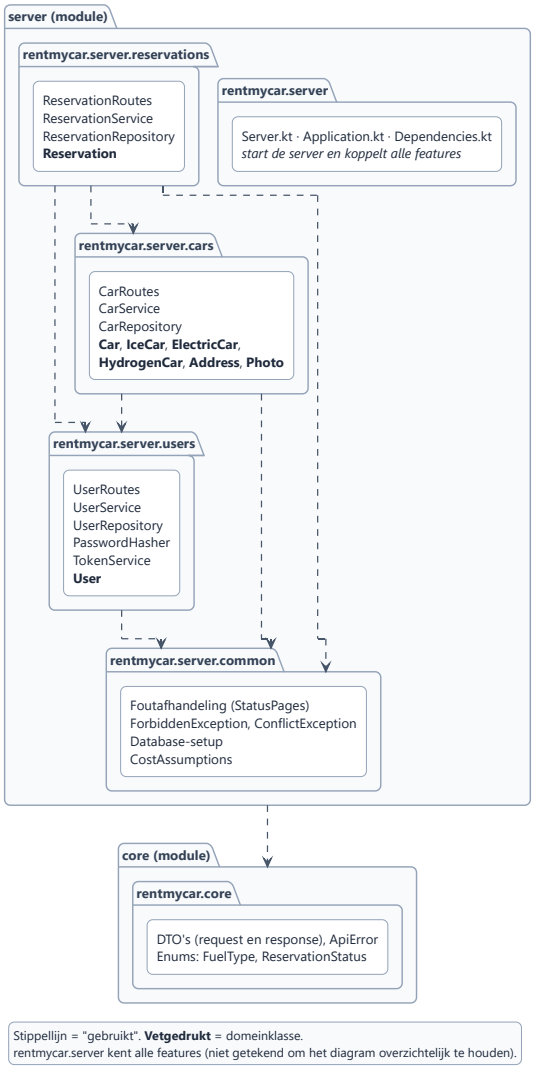
Wat	Stijl	Naam van de stijl	Voorbeeld
Klasse, interface, object, enum	Elk woord met hoofdletter	PascalCase (UpperCamelCase)	`ReservationService`
Functie, variabele, property, parameter	Eerste woord klein, daarna hoofdletters	camelCase	`findByOwner`, `hourlyRate`
Constante (`const val`), enum-waarde	Hoofdletters met underscore	SCREAMING_SNAKE_CASE	`KM_PER_YEAR`, `APPROVED`
Package	Kleine letters, punten	lowercase	`rentmycar.server.cars`
Databasekolom/-tabel	Kleine letters met underscore	snake_case	`hourly_rate`, `owner_id`
Bestand	Naam van de (hoofd)klasse	PascalCase + `.kt`	`CarService.kt`
URL-pad	Kleine letters, meervoud	REST-stijl	`/api/cars/{id}`

Functienamen beginnen met een werkwoord en zeggen wat ze doen: `findById` (zoeken, mag niets vinden → nullable), `getActiveCar` (ophalen, gooit 404 als het er niet is), `existsByLicensePlate` (ja/nee), `isValidEmail` (Boolean begint met `is` / `has`). Afspraak uit CLAUDE.md: **namen in het Engels**, commentaar Engels en kort, documentatie Nederlands.

3.5 De afhankelijkheden tussen de lagen

De pijlen gaan maar één kant op: Routes kennen de Service, de Service kent Domein en Repository-*interface*, en alleen de Exposed-repository kent de Tables. Het domein (`Car`, `Reservation`) kent geen Ktor en geen database. Daardoor kun je het domein testen zonder server of database (zoals `CarCostsTest` doet).

Packagediagram - Server en core



Packagediagram (docs/images/package-diagram.svg)

4. Gebruikte technieken en libraries

Een **library** (bibliotheek) is code van iemand anders die je gebruikt. Een **framework** is een library die de structuur van je programma bepaalt en jouw code aanroept (Ktor roept jouw routes aan). Een **dependency** (afhankelijkheid) is een library waar jouw project van afhangt; die staan in `module.yaml`.

Kotlin Toolchain (`kotlin.bat`, `module.yaml`)

Wat: het bouwgereedschap dat de code compileert, de libraries downloadt, de server start en de tests draait. Het alternatief is Gradle; dit project gebruikt bewust de nieuwere, eenvoudigere Kotlin Toolchain.

Commando's: `./kotlin.bat run -m server` (starten), `./kotlin.bat test -m server` (testen). `-m server` = module server.

Bestanden: `module.yaml` per module (`product: jvm/app` = een programma dat op de JVM draait; `kmp/lib` = een Kotlin Multiplatform-bibliotheek). `libs.versions.toml` = de *version catalog*: alle versies op één plek, zodat core en server dezelfde versie gebruiken.

JVM

Wat: Java Virtual Machine – het programma dat gecompileerde Kotlin- (en Java-)code uitvoert. Kotlin op de server wordt naar JVM-bytecode gecompileerd. Daardoor kun je ook alle Java-libraries gebruiken (zoals BouncyCastle en java-jwt).

Ktor (versie 3.6.0)

Wat: een web-framework van JetBrains (de makers van Kotlin) om servers en HTTP-clients te bouwen. Het is in Kotlin geschreven en gebruikt coroutines.

Waarom: het lesmateriaal van de module gebruikt Ktor; het is licht en je kiest zelf welke onderdelen (**plugins**) je aanzet.

Gebruikte plugins: `ktor.server.core` (de basis), `netty` (de webserver-motor), `contentNegotiation` + `serialization.kotlinx.json` (JSON), `statusPages` (foutafhandeling), `auth` + `auth.jwt` (inloggen met tokens), `rateLimit` (beperken van pogingen). Voor tests: `testHost` (`testApplication`) en `client.contentNegotiation`.

Begrippen: *plugin* = een uitbreiding die je met `install(...)` aanzet. *call* = één verzoek + het antwoord daarop (`ApplicationCall`). *route* = pad + methode waarop code draait. *principal* = de ingelogde gebruiker na authenticatie.

Netty

Wat: de eigenlijke webserver die netwerkverbindingen afhandelt. Ktor kan verschillende "engines" gebruiken; Netty is de meest gebruikte. Jij merkt er alleen iets van in `embeddedServer(Netty, port = 8080)`.

kotlinx.serialization

Wat: de officiële Kotlin-library om objecten om te zetten naar JSON (**serialiseren**) en terug (**deserialiseren**).

Hoe: zet `@Serializable` boven een klasse; een compiler-plugin (aangezet met `serialization: json` in `module.yaml`) maakt de omzetcode. Ktor gebruikt het via ContentNegotiation.

Waarom niet Jackson/Gson (Java)? Kotlin-library heeft de voorkeur (CLAUDE.md), werkt ook op Android/multiplatform en snapt Kotlin-dingen zoals nullable types en default-waarden.

kotlinx.datetime

Wat: Kotlin-library voor datum en tijd: `LocalDateTime` (datum + tijd zonder tijdzone), `LocalTime`, `TimeZone`, `DateTimeUnit`. Samen met `kotlin.time.Clock` en `Instant`.

Waarom: werkt in de multiplatform-module `core` (ook op Android), in plaats van `java.time`. In `core/module.yaml` staat hij als `exported`: wie `core` gebruikt, krijgt deze library automatisch mee.

Exposed (versie 1.5.0)

Wat: een Kotlin-library van JetBrains om met een SQL-database te werken. Je beschrijft tabellen als Kotlin-`object` (`CarTable`) en schrijft queries in Kotlin: `CarTable.selectAll().where { CarTable.id eq id }`.

Soort: Exposed heeft twee stijlen: de **DSL** (lijkt op SQL – deze gebruikt dit project) en de **DAO**-stijl (objecten die zichzelf opslaan). De DSL is explicieter: je ziet precies welke query er draait.

Waarom: geen SQL-strings, dus geen SQL-injectie en typefouten worden door de compiler gevonden. Modules: `exposed.core`, `exposed.jdbc` (verbinding via JDBC), `exposed.kotlin.datetime` (kolommen van het type `LocalDateTime`).

JDBC = de standaard Java-manier om met databases te praten; de "URL" `jdbc:h2:...` zegt welke database.

H2 (versie 2.4.240)

Wat: een kleine SQL-database die als library in je programma meedraait. Geen installatie of aparte server nodig.

Twee standen: *file* (opgeslagen in `./data/rentmycar.mv.db`) voor de echte server; *mem* (alleen in het geheugen) voor tests.

Waarom: eenvoudig voor een schoolproject; past bij het lesmateriaal. Voor productie zou je bijvoorbeeld PostgreSQL nemen; door de repository-interface hoeft dan alleen de database-code te veranderen.

BouncyCastle (`bcprov-jdk18on` 1.84)

Wat: een bekende cryptografie-library voor Java. Hier alleen gebruikt voor het Argon2id-algoritme (`Argon2BytesGenerator`), omdat Java/Kotlin zelf geen Argon2 heeft.

java-jwt (Auth0)

Wat: de library die tokens maakt en controleert (`JWT.create()`, `JWT.require()`). Je ziet hem niet in `module.yaml`, omdat de Ktor-plugin `auth.jwt` hem zelf meebrengt (een **transitieve dependency**).

Logback

Wat: de library die logregels (meldingen van Ktor en `log.error(...)`) naar de console schrijft. Instellingen in `server/resources/logback.xml`.

kotlin.test

Wat: de standaard testbibliotheek van Kotlin: `@Test`, `assertEquals`, `assertTrue`, `assertIs`, `assertFailsWith`. Keuze uit het ontwerp in plaats van Kotest (die de gastdocent gebruikt), omdat hij eenvoudiger is (staat in de AI-log).

5. Begrippen van een Web API (HTTP, REST, JSON)

5.1 Client en server

De **server** (dit project) wacht op verzoeken. Een **client** (later de Android-app, nu IntelliJ's `.http`-bestanden of de tests) stuurt een **request** en krijgt een **response**. De server draait op `http://localhost:8080`: *localhost* = je eigen computer, *8080* = het poortnummer. `host = "0.0.0.0"` betekent: luister op alle netwerkkaarten (dus ook bereikbaar vanaf een telefoon of emulator in hetzelfde netwerk).

5.2 Opbouw van een HTTP-request en -response

```
POST /api/cars HTTP/1.1                          ← methode + pad
Host: localhost:8080                             ← headers
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json
                                                 ← lege regel
{ "type": "BEV", "brand": "Tesla", ... }         ← body

HTTP/1.1 201 Created                             ← statuscode
Content-Type: application/json

{ "id": 8, "type": "BEV", "brand": "Tesla", ... }
```

Onderdeel	Uitleg	In de code
Methode	Wat je wilt doen (GET, POST, PUT, PATCH, DELETE)	`get { }`, `post { }`, …
Pad	Welke resource	`route("/cars")`, `"/{id}"`
Path parameter	Variabel deel van het pad: de 5 in `/api/cars/5`	`call.intParameter("id")`
Query parameter	Na het vraagteken: `?city=Breda&type=BEV`; voor filters	`call.request.queryParameters["city"]`
Header	Extra informatie, bijvoorbeeld het token	gelezen door de Authentication-plugin
Body	De data (JSON)	`call.receive<CarRequest>()`
Statuscode	Hoe het ging	`call.respond(HttpStatusCode.Created, ...)`

5.3 REST

REST is een manier om een API op te zetten: alles is een **resource** met een eigen URL (`/api/cars/5` = auto 5), en de HTTP-methode zegt wat je ermee doet. Namen van resources zijn zelfstandige naamwoorden in het meervoud (`cars`, `reservations`), geen werkwoorden (dus niet `/api/createCar`). REST is **stateless**: de server onthoudt niets tussen twee verzoeken; daarom stuurt de client bij elk verzoek het token mee.

Methode	Betekenis	Body?	Idempotent?	Hier gebruikt voor
GET	Ophalen	nee	ja	auto's zoeken, reservering bekijken
POST	Iets nieuws maken	ja	nee	registreren, inloggen, auto toevoegen, reservering aanvragen
PUT	Heel object vervangen	ja	ja	auto wijzigen (alle velden worden meegestuurd)
PATCH	Deel wijzigen	ja	nee	alleen de status van een reservering
DELETE	Verwijderen	nee	ja	auto (soft delete)

Idempotent = twee keer hetzelfde verzoek sturen heeft hetzelfde effect als één keer. Twee keer dezelfde PUT laat dezelfde auto achter; twee keer POST maakt twee auto's.

Waarom is login een POST en geen GET? Het wachtwoord moet in de body, niet in de URL (URL's komen in logbestanden en de browsergeschiedenis).

5.4 Statuscodes in dit project

Code	Naam	Wanneer hier
200	OK	Gelukt, met data terug (standaard bij `call.respond(obj)`)
201	Created	Iets nieuws gemaakt (registreren, auto, reservering)
204	No Content	Gelukt, niets terug te sturen (auto verwijderd)
400	Bad Request	Foute invoer (validatie, kapotte JSON, "abc" als id)
401	Unauthorized	Niet (goed) ingelogd; ook bij fout wachtwoord
403	Forbidden	Ingelogd, maar niet van jou
404	Not Found	Bestaat niet (of auto is inactief)
409	Conflict	Botst met de huidige toestand (dubbel kenteken, overlap, verkeerde status)
429	Too Many Requests	Meer dan 10 inlog-/registratiepogingen per minuut
500	Internal Server Error	Onverwachte fout in de server

Vuistregel: **2xx** = gelukt, **4xx** = de client deed iets fout, **5xx** = de server deed iets fout.

5.5 JSON

JSON (JavaScript Object Notation) is het tekstformaat van de data. `{ }` = een object (wordt een Kotlin-klasse), `[ ]` = een lijst (`List`), `"tekst"` = `String`, getallen = `Int` / `Double`, `true` / `false` = `Boolean`, `null` = afwezig (nullable). Een enum wordt tekst (`"BEV"`), een `LocalDateTime` wordt `"2030-01-10T09:00"` (ISO-8601).

5.6 Authenticatie en autorisatie

Authenticatie = vaststellen *wie* je bent (inloggen, token controleren → anders 401). **Autorisatie** = bepalen *wat* je mag (eigenaar? → anders 403). In dit project: authenticatie door de Ktor-plugin, autorisatie in de services.

6. Opstarten: van `main()` tot draaiende server

6.1 `Server.kt`

```
private const val DEVELOPMENT_SECRET = "only-for-local-development-change-me-please"

fun main() {
    val jwtSecret = System.getenv("JWT_SECRET") ?: DEVELOPMENT_SECRET
    if (jwtSecret == DEVELOPMENT_SECRET) {
        println("Warning: JWT_SECRET is not set, the development secret is used.")
    }

    val dependencies = createDependencies(DatabaseFactory.FILE_URL, jwtSecret)
    addDemoDataWhenEmpty(dependencies)

    embeddedServer(Netty, port = 8080, host = "0.0.0.0") {
        module(dependencies)
    }.start(wait = true)
}
```

- `System.getenv("JWT_SECRET") ?: DEVELOPMENT_SECRET` – de **Elvis-operator** `?:`: "als links `null` is, neem dan rechts". `getenv` geeft `String?` terug (kan null zijn).
- `private const val` – een constante die al tijdens het compileren bekend is, alleen zichtbaar in dit bestand.
- `embeddedServer(Netty, ...) { module(dependencies) }` – start een webserver (Netty is de motor). Het blok `{ ... }` is een **lambda** die de applicatie configureert.
- `start(wait = true)` – blijf draaien tot de server gestopt wordt. `wait = true` is een **named argument**.

Waarom het geheim uit een omgevingsvariabele? Een geheim hoort niet in de code (die staat op GitHub). Het ontwikkel-geheim is alleen een vangnet op je eigen computer en geeft een waarschuwing.

6.2 `Dependencies.kt` – zelf koppelen, zonder framework

```
fun createDependencies(databaseUrl: String, jwtSecret: String): Dependencies {
    val database = DatabaseFactory.connect(databaseUrl)

    val userRepository = ExposedUserRepository(database)
    val carRepository = ExposedCarRepository(database)
    val reservationRepository = ExposedReservationRepository(database, carRepository,
userRepository)

    val passwordHasher = PasswordHasher()
    val tokenService = TokenService(jwtSecret)

    val userService = UserService(userRepository, passwordHasher, tokenService)
    val carService = CarService(carRepository, userRepository, reservationRepository)
    val reservationService = ReservationService(reservationRepository, carRepository,
userRepository)
    return Dependencies(...)
}
```

Dit heet **dependency injection met de hand** (constructor injection): elke klasse krijgt wat ze nodig heeft via de constructor, in plaats van het zelf te maken. De klasse `Dependencies` is alleen een "doos" met alle

onderdelen.

- De volgorde is belangrijk: eerst de database, dan repositories, dan services (die repositories nodig hebben).
- **Waarom geen framework (zoals Koin)?** Afspraak uit het ontwerp: zo eenvoudig mogelijk, en je ziet precies wat er gebeurt.
- De tests roepen dezelfde functie aan met een database in het geheugen en een test-geheim. Dat is het voordeel van parameters in plaats van vaste waarden.

6.3 `Application.kt` – de plugins

```
fun Application.module(dependencies: Dependencies) {
    install(ContentNegotiation) {
        json(Json { prettyPrint = true; ignoreUnknownKeys = true })
    }
    configureStatusPages()
    install(RateLimit) {
        register(RateLimitName("credentials")) {
            rateLimiter(limit = 10, refillPeriod = 1.minutes)
        }
    }
    install(Authentication) {
        jwt("auth-jwt") {
            realm = "rentmycar"
            verifier(dependencies.tokenService.verifier)
            validate { credential ->
                if (credential.payload.subject != null) JWTPrincipal(credential.payload) else
null
            }
            challenge { _, _ ->
                call.respond(HttpStatusCode.Unauthorized, ApiError("Log in first: ..."))
            }
        }
    }
    routing {
        route("/api") {
            userRoutes(dependencies.userService)
            carRoutes(dependencies.carService)
            reservationRoutes(dependencies.reservationService)
        }
    }
}
```

Plugin	Wat het doet
`ContentNegotiation` + `json`	Zet automatisch JSON om naar Kotlin-objecten (`call.receive<T>()`) en terug (`call.respond(obj)`). `ignoreUnknownKeys` : extra velden in de JSON geven geen fout. `prettyPrint` : leesbaar ingesprongen JSON.
`StatusPages`	Vangt exceptions en maakt er een nette statuscode met `ApiError` van (zie Foutafhandeling).
`RateLimit`	Maakt een limiet met de naam "credentials": maximaal 10 verzoeken per minuut. Wordt gebruikt bij login en registreren.
`Authentication` + `jwt`	Controleert het token in de header `Authorization: Bearer ...` . `verifier` controleert handtekening en geldigheid; `validate` eist dat er een `subject` (gebruikers-id) in staat; `challenge` bepaalt het antwoord bij een ontbrekend of ongeldig token: 401.
`routing`	Alle routes onder `/api` .

Kotlin-concept: extension function. `fun Application.module(...)` voegt een functie toe aan de bestaande Ktor-klasse `Application` . Binnen de functie is `this` de applicatie, daarom kun je direct `install(...)` schrijven.

Kotlin-concept: lambda met ontvanger (DSL). Blokken zoals `install(RateLimit) { ... }` zijn lambda's. Ktor gebruikt ze om configuratie leesbaar te maken. `{ _, _ -> ... }` : de underscore betekent "deze parameter gebruik ik niet".

`1.minutes` komt uit `kotlin.time.Duration` : een extension property op `Int` .

6.4 `DatabaseFactory.kt`

```
object DatabaseFactory {
    const val FILE_URL = "jdbc:h2:file:./data/rentmycar"
    fun inMemoryUrl(name: String): String = "jdbc:h2:mem:$name;DB_CLOSE_DELAY=-1"

    fun connect(url: String): Database {
        val database = Database.connect(url, driver = "org.h2.Driver")
        transaction(database) {
            SchemaUtils.create(UserTable, CarTable, ReservationTable)
        }
        return database
    }
}
```

- `object` = een **singleton**: er bestaat precies één `DatabaseFactory` ; je roept hem aan als `DatabaseFactory.connect(...)` .
- **Bestand vs. geheugen**: de echte server bewaart in `./data/rentmycar` (blijft bewaard na herstart); tests gebruiken `mem:` (leeg bij elke start). `DB_CLOSE_DELAY=-1` : de geheugen-database blijft bestaan zolang het programma draait, ook als er even geen verbinding open is.
- `"...$name..."` = **string template**: de waarde van `name` wordt in de tekst gezet.
- `SchemaUtils.create` maakt de tabellen als ze nog niet bestaan.

6.5 `DemoData.kt`

Als `eva@example.com` nog niet bestaat, wordt de database gevuld: Eva en Sanne (eigenaren), Daan (huurder), 7 auto's van alle drie de soorten en 2 reserveringen van Daan (één goedgekeurd, één aangevraagd). Alle demo-gebruikers hebben wachtwoord `Welkom123!`.

```
val first = Reservation.request(golf, daan, inTwoDays, plusHours(inTwoDays, 4), 80, now)
first.approve(now)
reservations.add(first)
```

Let op: de demodata gaat via dezelfde domeinregels (`Reservation.request`, `approve`) en via de echte `PasswordHasher`. Er wordt dus niets "langs de regels heen" in de database gezet.

`private fun atTime(...)` en `plusHours(...)` zijn hulpfuncties die alleen in dit bestand zichtbaar zijn (`private` op bestandsniveau).

7. De module `core`: DTO's en enums

7.1 Enums

```
enum class CarType { ICE, BEV, FCEV }           // brandstof, batterij, waterstof
enum class FuelType { PETROL, DIESEL, LPG }
enum class ReservationStatus { REQUESTED, APPROVED, REJECTED, CANCELLED }
```

Een **enum** is een type met een vaste lijst waarden. Voordeel boven een `String`: je kunt geen tikfout maken ("APROVED"), en een `when` over een enum kan zonder `else`, omdat de compiler weet dat alle gevallen behandeld zijn.

7.2 DTO's (data classes met `@Serializable`)

Bestand	Klassen	Gebruikt bij
`UserDtos.kt`	`RegisterRequest`, `LoginRequest`, `TokenResponse`, `UserResponse`	/api/users/...
`CarDtos.kt`	`AddressDto`, `CarRequest`, `CarResponse`, `CostsResponse`	/api/cars/...
`ReservationDtos.kt`	`CreateReservationRequest`, `ChangeStatusRequest`, `OwnerContact`, `ReservationResponse`	/api/reservations/...
`ApiError.kt`	`ApiError(message)`	Elk foutantwoord

```
@Serializable
data class CarRequest(
    val type: CarType,
    val brand: String,
    ...
    val fuelType: FuelType? = null,
    val litersPer100Km: Double? = null,
    val kWhPer100Km: Double? = null,
    val kgPer100Km: Double? = null
)
```

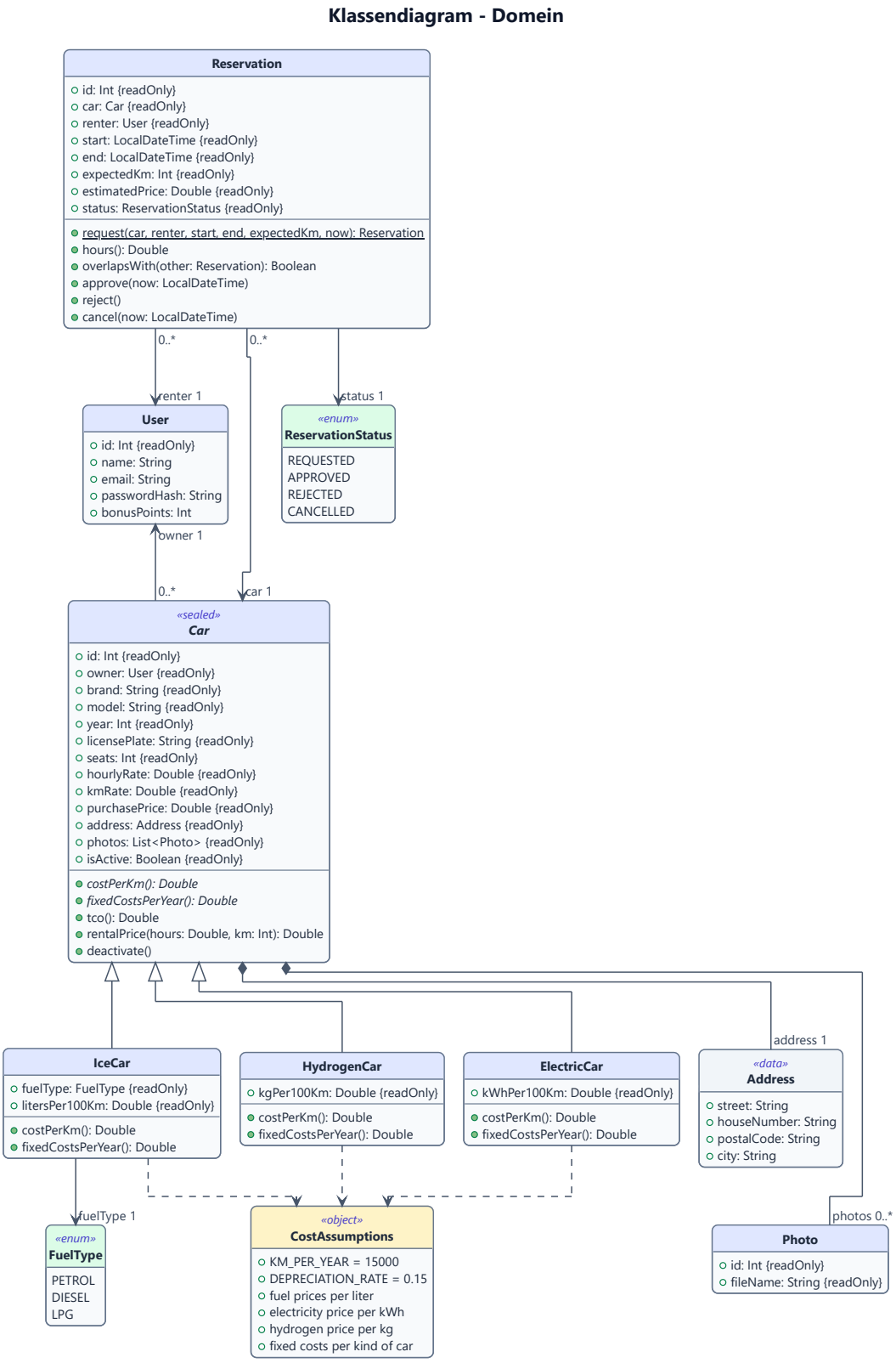
- `@Serializable` (kotlinx.serialization): de compiler maakt code om dit object naar JSON en terug te zetten. Daarom staat `serialization: json` in `module.yaml`.
- `data class`: Kotlin maakt automatisch `equals()`, `hashCode()`, `toString()` en `copy()`. Ideaal voor "zakjes data". De tests gebruiken `copy()`: `golfRequest().copy(hourlyRate = 7.5)`.
- `Double? = null`: het veld mag ontbreken (nullable met standaardwaarde). Alleen het verbruik dat bij het type hoort hoeft ingevuld te worden; de service controleert dat.
- `LocalDateTime` komt uit `kotlinx.datetime` (niet `java.time`), zodat het ook op Android en in de multiplatform-module werkt. In JSON wordt het tekst zoals `"2030-01-10T09:00"`.

7.3 Waarom aparte DTO's en domeinklassen?

- **Veiligheid**: `UserResponse` heeft géén wachtwoord-hash. Het domeinobject `User` wel. Zo kan de hash nooit per ongeluk in een antwoord terechtkomen.
- **Vorm**: het domein heeft een hele `User owner`, de DTO alleen `ownerId`.

- **Privacy**: `ReservationResponse.ownerContact` wordt pas gevuld na goedkeuring (zie het hoofdstuk Services).

8. Het domein: User, Car en Reservation



Figuur 2 – Domeinklassendiagram (docs/images/domain-class-diagram.svg)

8.1 `User`

```
class User(
    val id: Int,
    var name: String,
    var email: String,
    var passwordHash: String,
    var bonusPoints: Int = 0
)
```

- `val` = alleen-lezen (kan na het maken niet veranderen); `var` = mag veranderen. Het `id` verandert nooit, de rest kan veranderen.
- **Gewone class, geen data class**: een gebruiker heeft een eigen identiteit (id). Bovendien zou de automatische `toString()` van een data class de wachtwoord-hash in logbestanden kunnen zetten (dit staat ook in de AI-log).
- `bonusPoints: Int = 0`: **default parameter**.

8.2 `Address`

Wél een `data class`: een adres heeft geen eigen id; twee adressen met dezelfde velden zijn gewoon gelijk (*value object*).

8.3 `Car` – sealed class met drie soorten

```
sealed class Car(
    val id: Int, val owner: User, val brand: String, ...,
    val address: Address,
    isActive: Boolean = true
) {
    var isActive: Boolean = isActive
        private set

    init {
        require(brand.isNotBlank()) { "Brand is required." }
        require(seats > 0) { "A car must have at least 1 seat." }
        require(hourlyRate >= 0) { "Hourly rate cannot be negative." }
        ...
    }

    abstract fun costPerKm(): Double
    abstract fun fixedCostsPerYear(): Double

    fun tco(): Double =
        purchasePrice * CostAssumptions.DEPRECIATION_RATE +
            fixedCostsPerYear() +
            CostAssumptions.KM_PER_YEAR * costPerKm()

    fun rentalPrice(hours: Double, km: Int): Double = hours * hourlyRate + km * kmRate

    fun deactivate() { isActive = false }

    companion object { const val NEW_ID = 0 }
}
```

De concepten één voor één

Concept	In deze code	Waarom
sealed class	`Car` kan alleen `IceCar`, `ElectricCar` of `HydrogenCar` zijn (subklassen in hetzelfde package/module).	Bij `when (car) { is IceCar -> ...; is ElectricCar -> ...; is HydrogenCar -> ... }` weet de compiler dat er geen vierde soort is: geen `else` nodig. Voeg je een soort toe, dan geeft de compiler overal een fout waar je hem vergeet.
abstract fun	`costPerKm()` en `fixedCostsPerYear()` hebben geen inhoud in `Car`.	Elke soort berekent dit anders; de subklasse **moet** ze invullen met `override`.
Polymorfisme	`tco()` staat in `Car` maar roept `costPerKm()` aan.	Welke versie draait hangt af van het echte object. Eén formule voor alle soorten.
init + require	Controles bij het maken van het object.	Een ongeldige auto kan niet eens bestaan. `require` gooit `IllegalArgumentException` → 400.
private set	`isActive` is van buiten te lezen, maar alleen `Car` zelf kan hem veranderen (via `deactivate()`).	**Encapsulatie**: niemand kan een auto "per ongeluk" weer actief maken.
Constructorparameter zonder val	`isActive: Boolean = true` (zonder `val`) wordt alleen gebruikt om de property te vullen.	Zo kan de property een `private set` krijgen.
companion object	`Car.NEW_ID = 0`	Een "statische" waarde die bij de klasse hoort. Id 0 betekent: nog niet opgeslagen; de database geeft het echte id.
Expression body	`fun rentalPrice(...) = hours * hourlyRate + km * kmRate`	Korte schrijfwijze als de functie één expressie teruggeeft.

Ontwerpkeuze: alles `val`

De gegevens van een auto zijn `val`. Bij het wijzigen (PUT) maakt de service een **nieuw** `Car`-object met hetzelfde id. Daardoor worden alle controles in `init` opnieuw uitgevoerd. Alleen `isActive` kan veranderen. (Voorstel van de AI, door de student goedgekeurd – zie AI-log.)

8.4 De drie subklassen

```
class IceCar(
    id: Int, owner: User, ..., isActive: Boolean = true,
    val fuelType: FuelType,
    val litersPer100Km: Double
) : Car(id, owner, brand, model, year, licensePlate, seats,
        hourlyRate, kmRate, purchasePrice, address, isActive) {

    init { require(litersPer100Km > 0) { "Fuel use must be more than 0." } }

    override fun costPerKm(): Double =
        litersPer100Km * CostAssumptions.fuelPrice(fuelType) / 100

    override fun fixedCostsPerYear(): Double =
        CostAssumptions.ICE_INSURANCE + CostAssumptions.ICE_ROAD_TAX +
CostAssumptions.ICE_MAINTENANCE
}
```

`: Car(...)` betekent "erft van `Car` en geeft deze waarden door aan de constructor van `Car`". De parameters zonder `val` zijn alleen doorgeefluik; de eigen velden (`fuelType`, `litersPer100Km`) zijn wel `val`.

Klasse	Eigen veld	costPerKm()	Vaste kosten/jaar
`IceCar`	`fuelType`, `litersPer100Km`	liters × prijs per liter / 100	800 + 1000 + 650 = 2450
`ElectricCar`	`kWhPer100Km`	kWh × 0,34 / 100	1100 + 710 + 385 = 2195
`HydrogenCar`	`kgPer100Km`	kg × 21,0 / 100	1100 + 710 + 600 = 2410

8.5 `CostAssumptions` en een rekenvoorbeeld

`object CostAssumptions` bevat alle vaste getallen uit `docs/aannames.md` als `const val` (15.000 km per jaar, 15% afschrijving, brandstofprijzen, verzekering enz.). `fuelPrice(fuelType)` geeft met `when` de literprijs.

Rekenvoorbeeld: Volkswagen Golf, benzine, 6 l/100 km, € 30.000 (dit test UT-01 en UT-03)
costPerKm = 6 × 2,42 / 100 = **€ 0,1452**
TCO = afschrijving 30.000 × 0,15 = 4500
+ vaste kosten 800 + 1000 + 650 = 2450
+ brandstof 15.000 × 0,1452 = 2178
= **€ 9128 per jaar**
Huurprijs 4 uur + 80 km = 4 × 6,00 + 80 × 0,25 = **€ 44,00**

8.6 `Reservation`

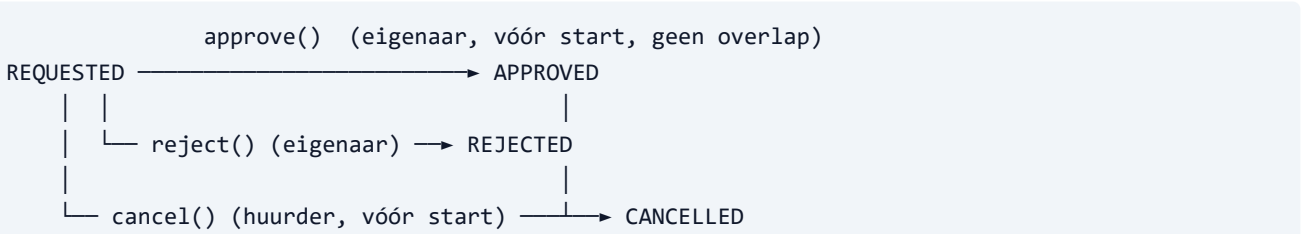
```
class Reservation(
    val id: Int, val car: Car, val renter: User,
    val start: LocalDateTime, val end: LocalDateTime,
    val expectedKm: Int, val estimatedPrice: Double,
    status: ReservationStatus = ReservationStatus.REQUESTED
) {
    var status: ReservationStatus = status
        private set

    fun overlapsWith(other: Reservation): Boolean =
        start < other.end && other.start < end

    fun approve(now: LocalDateTime) {
        check(status == ReservationStatus.REQUESTED) { "Only a requested reservation can be
approved." }
        check(start > now) { "This reservation has already started." }
        status = ReservationStatus.APPROVED
    }
    fun reject() { ... }        // alleen vanuit REQUESTED
    fun cancel(now: LocalDateTime) { ... } // vanuit REQUESTED of APPROVED, vóór de start

    companion object {
        fun request(car, renter, start, end, expectedKm, now): Reservation {
            require(renter.id != car.owner.id) { "You cannot reserve your own car." }
            require(start > now) { "The start must be in the future." }
            require(end > start) { "The end must be after the start." }
            require(hoursBetween(start, end) >= 1.0) { "A reservation must last at least 1 hour."
}
            require(expectedKm >= 0) { "Expected km cannot be negative." }
            val price = car.rentalPrice(hoursBetween(start, end), expectedKm)
            return Reservation(NEW_ID, car, renter, start, end, expectedKm, price)
        }
    }
}
```

Statusovergangen (toestandsmachine)



- **`require` vs `check`**: `require` controleert de *invoer* (argumenten) en gooit `IllegalArgumentException` → **400**. `check` controleert de *toestand* van het object en gooit `IllegalStateException` → **409**. "Jouw invoer klopt niet" versus "het kan nu niet".
- **Factory-functie** `Reservation.request(...)` in het `companion object`: een nieuwe aanvraag maak je altijd via deze functie, zodat alle regels gecontroleerd worden en de prijs wordt vastgelegd. De gewone constructor wordt gebruikt bij het teruglezen uit de database (die gegevens zijn al gecontroleerd).
- **De prijs wordt vastgelegd** (`estimatedPrice`) op het moment van aanvragen. Als de eigenaar later het tarief verandert, blijft de prijs van de reservering gelijk.

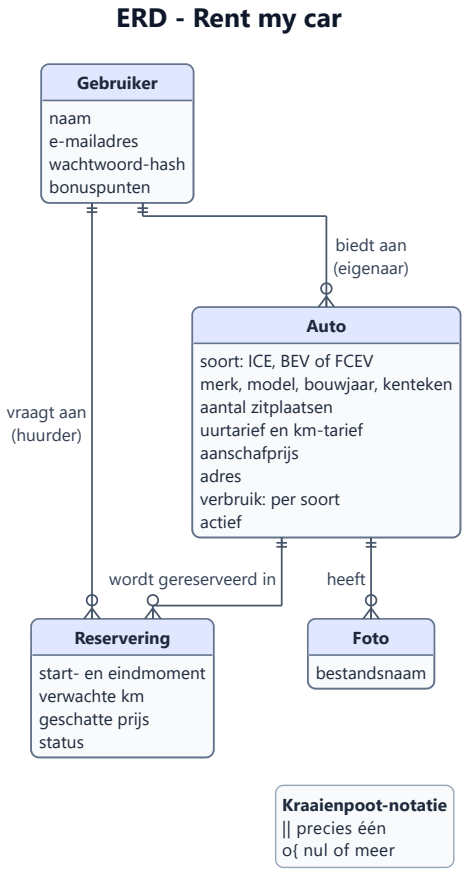
- **Overlap**: twee perioden overlappen als elk begint vóór het andere eindigt. 9–12 en 12–15 overlappen dus *niet* (de één eindigt precies als de ander begint).
- `now` **als parameter**: de functie vraagt niet zelf de klok op. In de tests geef je een vaste tijd mee (`NOW = 2030-01-01 12:00`), zodat de test altijd hetzelfde resultaat geeft.

8.7 Hulpfuncties in `common`

Functie	Uitleg
`currentTime()`	Huidige tijd als `LocalDateTime` in de tijdzone van de server (`kotlin.time.Clock`).
`hoursBetween(start, end)`	Zet beide om naar een `Instant` (in UTC, zodat zomertijd geen rol speelt), trekt ze af, neemt hele minuten en deelt door 60.0 → bijvoorbeeld 2.5 uur.
`Double.rounded(decimals)`	Extension function: `0.1452.rounded(2)` = 0.15. Factor = 10^{decimals} via `repeat`; dan `round(x × factor) / factor`.

9. Opslag: tabellen en repositories (Exposed + H2)

H2 is een kleine database die in Java draait (geen aparte installatie). **Exposed** is een Kotlin-library van JetBrains om met SQL te werken in Kotlin-code (DSL-stijl), in plaats van SQL-strings te schrijven.



Figuur 3 – ERD (docs/images/erd.svg)

9.1 Tabellen

```
object UserTable : IntIdTable("users") {
    val name = varchar("name", 100)
    val email = varchar("email", 255).uniqueIndex()
    val passwordHash = varchar("password_hash", 255)
    val bonusPoints = integer("bonus_points").default(0)
}
```

- `IntIdTable` geeft automatisch een kolom `id` die zelf ophoogt (auto increment).
- `.uniqueIndex()`: de database staat geen twee dezelfde e-mailadressen (of kentekens bij `CarTable`) toe. Een extra vangnet naast de controle in de service.
- `reference("owner_id", UserTable)`: een **foreign key** naar de tabel users.
- `enumerationByName<CarType>("type", 10)`: de enum wordt als tekst ("ICE") opgeslagen, niet als nummer. Leesbaar, en de volgorde van de enum maakt niet uit.
- `.nullable()`: de kolom mag leeg zijn.
- In `ReservationTable` heten de kolommen `start_time` / `end_time`, omdat `END` een gereserveerd woord in SQL is.

Eén tabel voor drie soorten auto's (single table inheritance)
Alle auto's staan in de tabel `cars`. De kolom `type` zegt welke soort het is. Kolommen die maar bij één soort horen (`fuel_type`, `liters_per_100km`, `kwh_per_100km`, `kg_per_100km`) zijn `nullable`. Voordeel: eenvoudig, geen joins nodig. Nadeel: lege kolommen. (Zie hoofdstuk 2.5 van het ontwerp.)

9.2 Repository: interface + Exposed-implementatie

```
interface CarRepository {
    fun findById(id: Int): Car?
    fun findAllActive(): List<Car>
    fun findByOwner(ownerId: Int): List<Car>
    fun existsByLicensePlate(licensePlate: String): Boolean
    fun add(car: Car): Car
    fun update(car: Car)
}

class ExposedCarRepository(private val database: Database) : CarRepository {
    override fun findById(id: Int): Car? = transaction(database) {
        (CarTable innerJoin UserTable).selectAll()
            .where { CarTable.id eq id }
            .map { toCar(it) }
            .singleOrNull()
    }
    ...
}
```

- Een **interface** beschrijft *wat* een repository kan, niet *hoe*. `ExposedCarRepository` is de echte versie; `FakeCarRepository` (in de tests) bewaart alles in een lijst.
- `Car?`: **nullable return type**. Niet gevonden = `null`; de service beslist wat dan (404).
- `transaction(database) { ... }`: alle databasewerk gebeurt binnen een transactie. De laatste expressie van het blok is de returnwaarde.
- `innerJoin UserTable`: haalt de eigenaar in dezelfde query op, zodat er meteen een volledig `User`-object gemaakt kan worden. `innerJoin` wordt als **infix-functie** geschreven (zonder punt en haakjes).
- `.map { toCar(it) }`: elke rij omzetten; `it` is de standaardnaam van de enige lambda-parameter.
- `.singleOrNull()`: precies één resultaat, of `null` als er geen is.

9.3 Van object naar rij en terug

`writeCar(row, car)` zet de waarden in de kolommen. Eerst worden alle soort-specifieke kolommen leeg gemaakt (`null`), daarna met `when (car) { is IceCar -> ... }` de juiste gevuld. Binnen zo'n tak is `car` automatisch een `IceCar`: **smart cast**, daarom mag je `car.fuelType` schrijven.

`toCar(row)` doet het omgekeerde: op basis van `row[CarTable.type]` wordt het juiste soort object gemaakt. `row[CarTable.fuelType]!!`: de **not-null assertion** `!!` zegt "ik weet zeker dat dit niet null is" – bij een ICE-auto is die kolom altijd gevuld, omdat `writeCar` dat zo opslaat.

`add(car)` slaat op met `insertAndGetId` en leest de auto daarna terug met `findById`, zodat je het object mét het echte id krijgt.

9.4 `ExposedReservationRepository`

Deze krijgt naast de database ook de `CarRepository` en `UserRepository`, om bij elke reservering de volledige auto en huurder te laden. `findApprovedForCar` gebruikt `(car eq carId) and (status eq APPROVED)`. `update` wijzigt alleen de status, omdat dat het enige is wat bij een reservering kan veranderen.

Opvallend: `transaction` geeft eerst de rijen terug (`.toList()`), en pas daarna wordt `toReservation` aangeroepen (buiten de transactie). Dat komt doordat `toReservation` zelf weer repositories aanroept die hun eigen transactie openen.

10. Services: de use cases en de regels

10.1 `UserService`

```
fun register(request: RegisterRequest): User {
    require(request.name.isNotBlank()) { "Name is required." }
    require(isValidEmail(request.email)) { "Email address is not valid." }
    require(request.password.length >= 8) { "Password must have at least 8 characters." }
    if (userRepository.findByEmail(request.email) != null) {
        throw ConflictException("This email address is already in use.")
    }
    val passwordHash = passwordHasher.hash(request.password)
    return userRepository.add(request.name, request.email, passwordHash)
}

fun login(request: LoginRequest): String? {
    val user = userRepository.findByEmail(request.email) ?: return null
    if (!passwordHasher.verify(request.password, user.passwordHash)) return null
    return tokenService.createToken(user.id)
}
```

- `?: return null`: Elvis met een `return` – als de gebruiker niet bestaat, stop dan meteen.
- **Zelfde antwoord bij verkeerd e-mailadres en verkeerd wachtwoord**: zo kan een aanvaller niet uitzoeken welke e-mailadressen bestaan (user enumeration, OWASP).
- Het wachtwoord wordt nooit opgeslagen, alleen de hash.
- `isValidEmail` gebruikt een eenvoudige `Regex`.

10.2 `CarService`

Functie	Wat en welke regels
`search(type, city, maxHourlyRate)`	Alle actieve auto's, dan stap voor stap filteren met `filter { }`. Elk filter is optioneel (`null` = niet filteren). Stad zonder hoofdlettergevoeligheid (`ignoreCase = true`).
`getActiveCar(id)`	404 als de auto niet bestaat *of* inactief is.
`getOwnCar(id, userId)`	Als `getActiveCar`, plus **403** als jij niet de eigenaar bent. Wordt gebruikt bij wijzigen, verwijderen en kosten.
`createCar(request, userId)`	409 bij dubbel kenteken. De ingelogde gebruiker wordt eigenaar (niet een id uit de JSON!).
`updateCar(...)`	Eigenaar-check, kenteken-check (alleen als het kenteken verandert), dan een nieuw object met hetzelfde id.
`deleteCar(id, userId, now)`	**Soft delete**: `deactivate()` in plaats van echt verwijderen (oude reserveringen blijven kloppen). 409 als er nog open reserveringen in de toekomst zijn (`any { ... }`).
`getCosts(id, userId)`	Alleen voor de eigenaar. Kosten per km op 4 decimalen, TCO op 2.
`makeCar(...)` (private)	Maakt met `when (request.type)` de juiste subklasse. Ontbreekt het verbruik dat bij het type hoort, dan `throw IllegalArgumentException` → 400. Let op `request.fuelType ?: throw ...`: `throw` is in Kotlin een expressie, dus kan rechts van `?:` staan.

Eigenaarschap in de service (403)
De route weet alleen *wie* er is ingelogd (`call.userId()`). De service beslist of die gebruiker iets *mag*. Dat staat zo in CLAUDE.md/het ontwerp, en voorkomt "Broken Object Level Authorization" (OWASP API Security Top 10, nummer 1): zonder deze check zou iedereen met een token de auto van een ander kunnen wijzigen door een ander id in de URL te zetten.

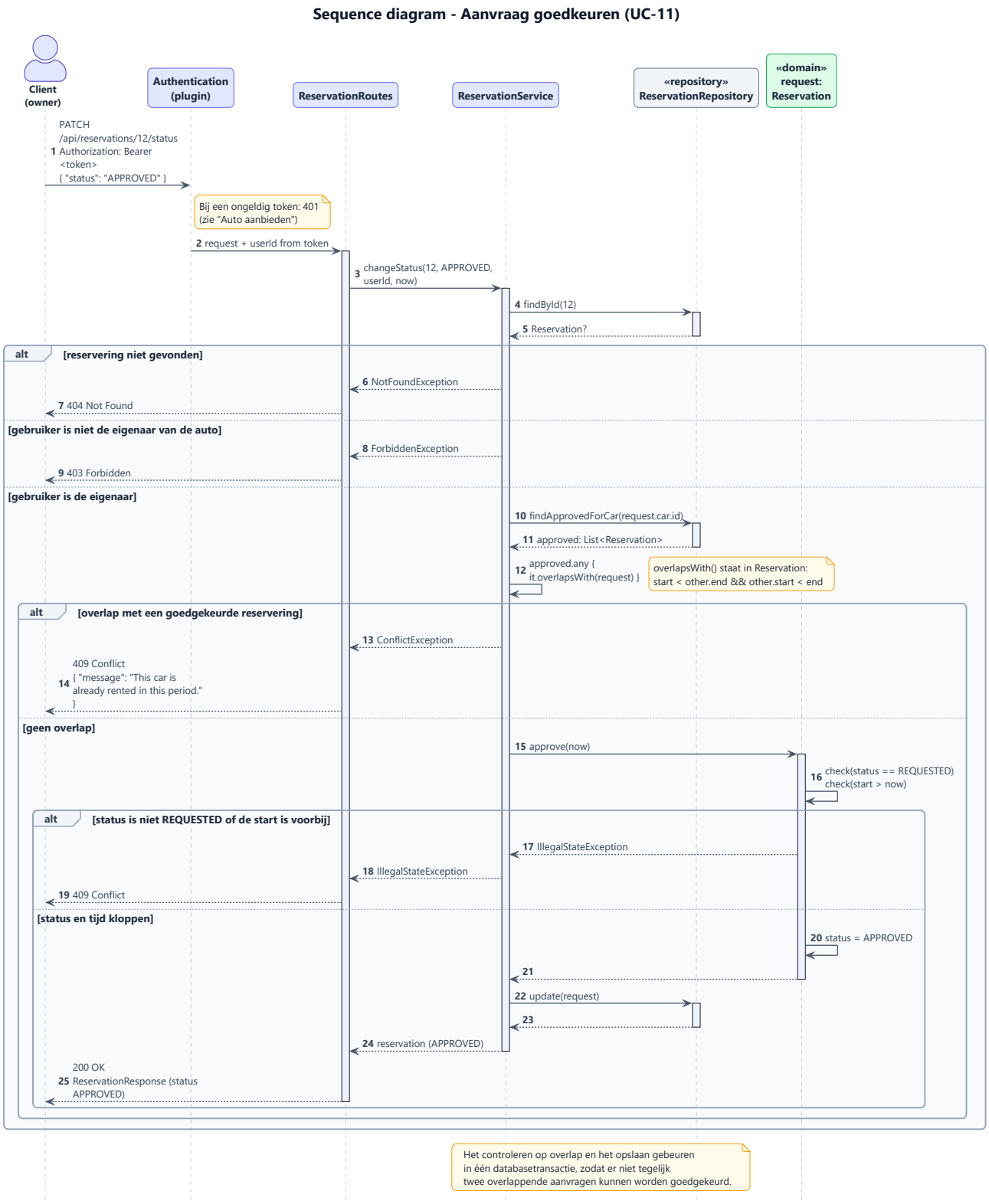
10.3 `ReservationService`

```
fun changeStatus(id: Int, newStatus: ReservationStatus, userId: Int, now: LocalDateTime):
Reservation {
    val reservation = findReservation(id)
    when (newStatus) {
        ReservationStatus.APPROVED -> approve(reservation, userId, now)
        ReservationStatus.REJECTED -> { checkIsOwner(reservation, userId); reservation.reject();
... }
        ReservationStatus.CANCELLED -> { /* alleen de huurder */ reservation.cancel(now); ... }
        ReservationStatus.REQUESTED -> throw IllegalArgumentException("A reservation cannot be
set back to requested.")
    }
    return reservation
}

private fun approve(reservation: Reservation, userId: Int, now: LocalDateTime) {
    checkIsOwner(reservation, userId)
    synchronized(approveLock) {
        val approved = reservationRepository.findApprovedForCar(reservation.car.id)
        if (approved.any { it.overlapsWith(reservation) }) {
            throw ConflictException("This car is already rented in this period.")
        }
        reservation.approve(now)
        reservationRepository.update(reservation)
    }
}
```

Wie mag wat?	Eigenaar van de auto	Huurder	Ander
Reservering bekijken	ja	ja	403
Goedkeuren / afwijzen	ja	403	403
Annuleren	403	ja	403
Reserveringen van een auto zien	ja	403	403

- **Waarom pas bij goedkeuren op overlap controleren?** Meerdere huurders mogen dezelfde periode *aanvragen*; de eigenaar kiest. Alleen twee *goedgekeurde* reserveringen mogen niet overlappen.
- `synchronized(approveLock)` : de server behandelt verzoeken tegelijk (meerdere threads). Zonder lock zouden twee goedkeuringen tegelijk allebei "geen overlap" kunnen zien en allebei opslaan (*race condition*). Met het lock kan maar één thread tegelijk dit blok uitvoeren. `approveLock = Any()` is gewoon een leeg object dat als "sleutel" dient. (Dit werkt omdat er één serverproces is.)
- **Contactgegevens pas na goedkeuring**: in `ReservationMapping.kt` wordt `ownerContact` alleen gevuld als de status `APPROVED` is (privacy / dataminimalisatie).
- `when` over de enum zonder `else`: de compiler dwingt af dat alle vier statussen behandeld zijn.



Figuur 4 – Sequentiediagram reservering goedkeuren

11. Routes: de HTTP-kant (Ktor)

11.1 Alle endpoints

Methode + pad	Login?	Service-functie	Succes
POST /api/users/register	nee (rate limit)	`register`	201 + UserResponse
POST /api/users/login	nee (rate limit)	`login`	200 + TokenResponse, of 401
GET /api/users/me	ja	`findById`	200
GET /api/cars?type=&city=&maxHourlyRate=	nee	`search`	200 + lijst
GET /api/cars/{id}	nee	`getActiveCar`	200
GET /api/cars/mine	ja	`getCarsOfOwner`	200
POST /api/cars	ja	`createCar`	201
PUT /api/cars/{id}	ja	`updateCar`	200
DELETE /api/cars/{id}	ja	`deleteCar`	204
GET /api/cars/{id}/costs	ja	`getCosts`	200
GET /api/cars/{id}/reservations	ja	`getReservationsForCar`	200
POST /api/reservations	ja	`requestReservation`	201
GET /api/reservations/mine	ja	`getMyReservations`	200
GET /api/reservations/{id}	ja	`getReservation`	200
PATCH /api/reservations/{id}/status	ja	`changeStatus`	200

Waarom welke methode? GET = lezen, POST = iets nieuws maken (201 Created), PUT = een heel object vervangen, PATCH = een deel wijzigen (alleen de status), DELETE = verwijderen (204 No Content, er is geen body).

11.2 Hoe een route eruitziet

```
fun Route.carRoutes(carService: CarService) {
    route("/cars") {
        get {                                   // GET /api/cars
            val parameters = call.request.queryParameters
            val type = parameters["type"]?.let { CarType.valueOf(it.uppercase()) }
            val city = parameters["city"]
            val maxHourlyRate = parameters["maxHourlyRate"]?.toDouble()
            val cars = carService.search(type, city, maxHourlyRate)
            call.respond(cars.map { it.toResponse() })
        }
        authenticate("auth-jwt") {
            post {                              // POST /api/cars
                val request = call.receive<CarRequest>()
                val car = carService.createCar(request, call.userId())
                call.respond(HttpStatusCode.Created, car.toResponse())
            }
            ...
        }
    }
}
```

- `fun Route.carRoutes(...)`: weer een extension function, nu op Ktor's `Route`. Elke feature levert zo zijn eigen routes, en `Application.kt` plakt ze samen.
- `authenticate("auth-jwt") { ... }`: alle routes binnen dit blok vereisen een geldig token. Zonder token → 401 (via `challenge`).
- `call.receive<CarRequest>()`: lees de JSON-body als `CarRequest`. `<CarRequest>` is een **generic type-parameter**. Klopt de JSON niet → `BadRequestException` → 400.
- `?.let { ... }`: **safe call** – alleen uitvoeren als de waarde niet `null` is; anders is het resultaat `null`.
- De route is dun: lezen → service aanroepen → `toResponse()` → antwoorden. Geen regels in de route.
- `delete` en reserveringsroutes geven `currentTime()` mee als `now`; zo blijft de service testbaar met een vaste tijd.

11.3 `CallUtils.kt`

```
fun ApplicationCall.userId(): Int {
    val principal = principal<JWTPrincipal>() ?: throw IllegalStateException("No logged-in
user.")
    return principal.payload.subject.toInt()
}

fun ApplicationCall.intParameter(name: String): Int =
    parameters[name]?.toIntOrNull() ?: throw BadRequestException("'$name' must be a number.")
```

`userId()` haalt het gebruikers-id uit het `sub`-veld (subject) van het token. Het id komt dus uit het ondertekende token, niet uit iets wat de client zelf kan kiezen. `intParameter` geeft 400 bij `/api/cars/abc` in plaats van een crash.

De **Mapping**-bestanden (`User.toResponse()`, `Car.toResponse()`, `Reservation.toResponse()`) zetten domein om naar DTO. `Car.toResponse()` maakt eerst de gedeelde velden en voegt dan met `response.copy(...)` het verbruik van de juiste soort toe.

12. Foutafhandeling: van exception naar statuscode

In de service en het domein gooi je gewoon een exception. De plugin **StatusPages** (`common/StatusPages.kt`) vangt die en stuurt de juiste statuscode met een `ApiError`-body. Zo hoeft geen enkele route zelf `try/catch` te doen.

Exception	Waar vandaan	Status	Voorbeeld
`BadRequestException`	Ktor (kapotte JSON), `intParameter`	400	"'id' must be a number."
`IllegalArgumentException`	`require(...)`, `makeCar`	400	"Hourly rate cannot be negative."
(geen/ongeldig token)	`challenge` in Application.kt	401	"Log in first: ..."
`ForbiddenException` (eigen)	services	403	"This is not your car."
`NotFoundException`	services	404	"Car not found."
`ConflictException` (eigen)	services	409	"This license plate is already in use."
`IllegalStateException`	`check(...)`	409	"Only a requested reservation can be approved."
(te veel pogingen)	RateLimit-plugin	429	"Too many attempts. Try again in a minute."
`Throwable` (al het andere)	onverwacht	500	"Something went wrong on the server."

```
// Exceptions.kt
class ForbiddenException(message: String) : Exception(message)
class ConflictException(message: String) : Exception(message)
```

- Eigen exceptions erven van `Exception` en geven de melding door.
- Bij 500 gaat het detail naar de **log** (`log.error`), niet naar de client: geen interne informatie (stacktrace) lekken (OWASP: security misconfiguration / improper error handling).
- `cause.message ?: "Invalid request."`: `message` kan null zijn, dan een standaardtekst.
- `status(HttpStatusCode.TooManyRequests) { ... }` reageert op een statuscode in plaats van op een exception.

13. Beveiliging

13.1 Wachtwoorden: Argon2id (`PasswordHasher.kt`)

```
fun hash(password: String): String {
    val salt = ByteArray(16)
    SecureRandom().nextBytes(salt)
    val hash = calculateHash(password, salt)
    return Base64.encode(salt) + ":" + Base64.encode(hash)
}

fun verify(password: String, storedHash: String): Boolean {
    val parts = storedHash.split(":")
    if (parts.size != 2) return false
    val salt = Base64.decode(parts[0])
    val expectedHash = Base64.decode(parts[1])
    val actualHash = calculateHash(password, salt)
    return MessageDigest.isEqual(expectedHash, actualHash)
}

// Argon2id, versie 1.3, 64 MB geheugen (65_536 KB), 3 iteraties, parallelisme 4, 32 bytes output
```

- **Argon2id** is het algoritme dat OWASP (Password Storage Cheat Sheet) als eerste aanraadt. Het is bewust traag en gebruikt veel geheugen, zodat raden met veel computers/GPU's duur wordt. De library is BouncyCastle. Volgens het commentaar in de code zijn de instellingen gelijk aan de demo van de gastdocent.
- **Salt**: 16 willekeurige bytes per gebruiker (`SecureRandom`, cryptografisch veilig). Twee gebruikers met hetzelfde wachtwoord krijgen daardoor een andere hash, en "rainbow tables" werken niet.
- Opslag als `"salt:hash"` in Base64 (tekst), zodat je bij `verify` dezelfde salt kunt gebruiken.
- **Vergelijken in constante tijd** (`MessageDigest.isEqual`): een gewone vergelijking stopt bij het eerste verschil, en aan de tijd zou een aanvaller iets kunnen afleiden (timing attack).
- `65_536`: de underscore is alleen voor de leesbaarheid.

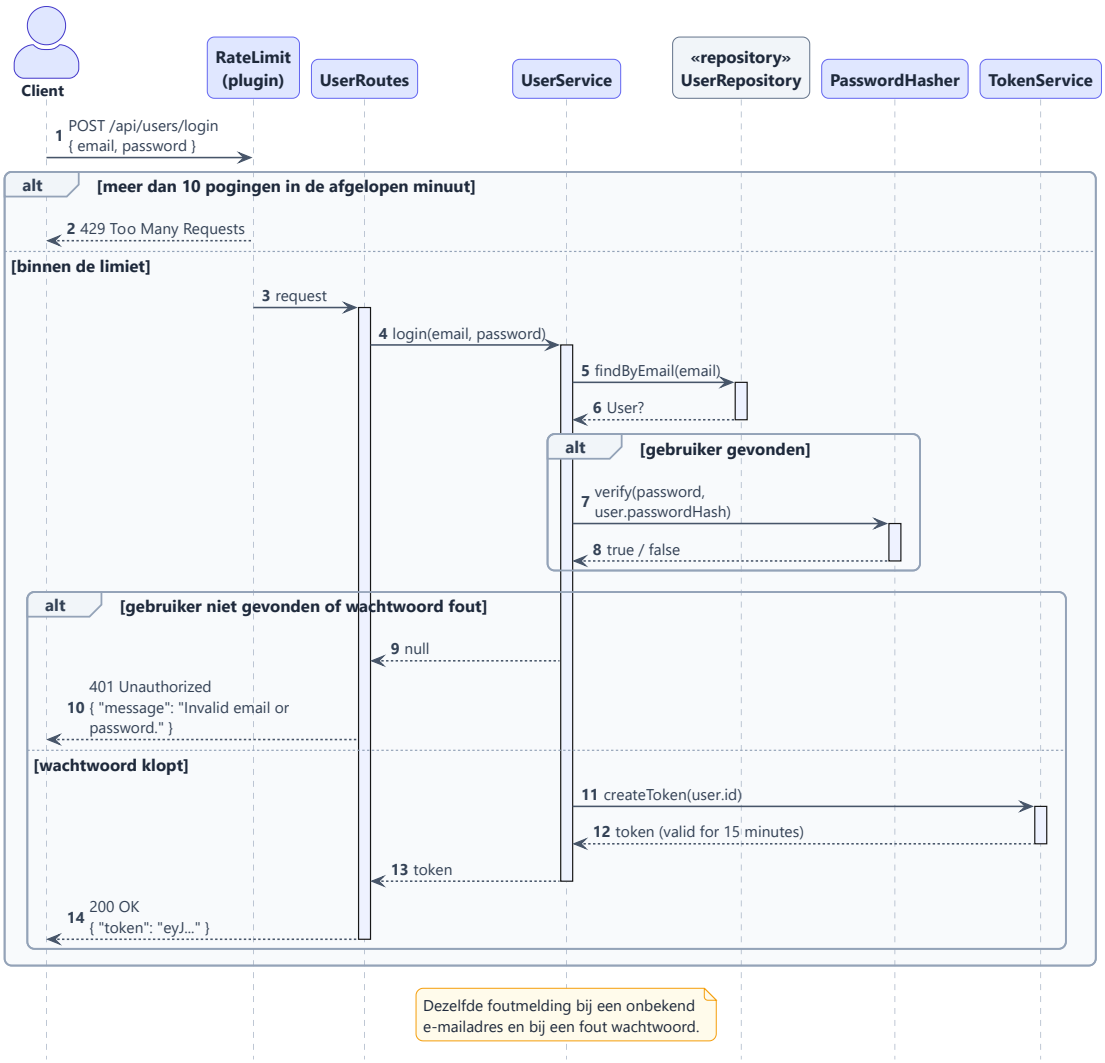
13.2 Tokens: JWT (`TokenService.kt`)

```
class TokenService(secret: String) {
    init { require(secret.length >= 32) { "The JWT secret must have at least 32 characters." } }
    private val algorithm = Algorithm.HMAC256(secret)

    val verifier: JWTVerifier = JWT.require(algorithm)
        .withIssuer(ISSUER).withAudience(AUDIENCE)
        .withClaimPresence("sub").withClaimPresence("exp")
        .build()

    fun createToken(userId: Int, now: Date = Date()): String {
        val expiresAt = Date(now.time + VALID_MINUTES * 60 * 1000)
        return JWT.create()
            .withIssuer(ISSUER).withAudience(AUDIENCE)
            .withSubject(userId.toString())
            .withIssuedAt(now).withExpiresAt(expiresAt)
            .sign(algorithm)
    }
    companion object { const val ISSUER = "rentmycar-server"; const val AUDIENCE = "rentmycar-
api"; const val VALID_MINUTES = 15 }
}
```

- Een JWT bestaat uit drie delen: *header.payload.handtekening*. De payload is **niet versleuteld** (alleen Base64), maar wel **ondertekend**: wie het token verandert, maakt de handtekening ongeldig. Zet er dus nooit geheimen in; hier staat alleen het gebruikers-id.
- **HMAC256** met een geheim van minimaal 32 tekens (256 bits). Een te kort geheim wordt geweigerd in `init`.
- Claims: `iss` (wie maakte het), `aud` (voor wie), `sub` (gebruikers-id), `iat` (gemaakt op), `exp` (verloopt). De verifier controleert ze allemaal.
- **15 minuten geldig**: als een token gestolen wordt, is het snel waardeloos.
- `now: Date = Date()`: standaard de huidige tijd, maar een test kan een eigen tijd meegeven.
- Hier wordt `java.util.Date` gebruikt, omdat de library java-jwt (die Ktor's JWT-plugin gebruikt) daarmee werkt. Dit is een bewuste uitzondering op "Kotlin-libraries waar mogelijk" (staat ook in het commentaar).



Figuur 5 – Sequentiediagram inloggen

13.3 Overzicht beveiligingsmaatregelen

Maatregel	Waar	Tegen
Argon2id + salt	PasswordHasher	Gestolen database → wachtwoorden achterhalen
Constante-tijd vergelijking	PasswordHasher.verify	Timing attacks
JWT, 15 min, iss/aud/exp gecontroleerd	TokenService, Application	Vervalste of gestolen tokens
Geheim uit omgevingsvariabele, min. 32 tekens	Server.kt, TokenService	Geheim in broncode, zwak geheim
Rate limiting 10/min	Application, UserRoutes	Wachtwoorden raden (brute force)
Zelfde fout bij login	UserService.login	Uitzoeken welke e-mails bestaan
Eigenaarschap in service (403)	CarService, ReservationService	Andermans data wijzigen (BOLA)
Id uit token, niet uit body	CallUtils.userId	Zich voordoen als een ander
DTO zonder hash; contact pas na goedkeuring	UserResponse, ReservationMapping	Datalekken
Generieke 500-melding	StatusPages	Interne details lekken
Exposed (geen SQL-strings)	Repositories	SQL-injectie
Validatie in domein (require)	Car, Reservation, UserService	Ongeldige data

14. Testen

Tests draaien met `./kotlin.bat test -m server`. Ze gebruiken `kotlin.test` (`@Test`, `assertEquals`, `assertIs`, `assertFailsWith`). Testnamen tussen backticks (`` `cost per km of a petrol car` ``) mogen spaties bevatten – handig voor leesbare namen.

Bestand	Soort	Wat wordt getest
`CarCostsTest`	Unit (UT-01, UT-03)	Kosten per km (0,1452) en TCO (9128) van de benzine-Golf.
`ReservationTest`	Unit (UT-05)	`overlapsWith` in vier gevallen: deels tegelijk, aansluitend, later, er helemaal binnen.
`ServiceTest`	Unit met fakes (UT-09)	Goedkeuren wordt geweigerd (409) bij overlap, en de status blijft REQUESTED.
`CarRepositoryTest`	Integratie (IT-01)	Echte Exposed-repository + H2 in geheugen: elke soort auto komt als dezelfde soort terug.
`ApiTest`	Integratie (IT-03, IT-04)	Hele app via HTTP: registreren, inloggen, CRUD op een auto; zonder token → 401.

14.1 Hulpbestanden

- `TestData.kt`: vaste testdata – `NOW` (2030-01-01 12:00), gebruikers `OWNER`, `RENTER`, `OTHER_USER`, `jan10(hour)`, en `petrolCar()`, `electricCar()`, `hydrogenCar()` met default parameters. Door een vaste "nu" in de toekomst slagen tests ook over jaren nog.
- `Fakes.kt`: `FakeUserRepository`, `FakeCarRepository`, `FakeReservationRepository` implementeren dezelfde interfaces, maar met een `MutableList`. Geen database nodig → snel en simpel. Dit kan alleen omdat de services van een *interface* afhangen.

14.2 Integratietest met `testApplication`

```
private fun apiTest(test: suspend (HttpClient) -> Unit) = testApplication {
    val databaseUrl = DatabaseFactory.inMemoryUrl("api-test-${System.nanoTime()}")
    val dependencies = createDependencies(databaseUrl, TEST_SECRET)
    application { module(dependencies) }
    val client = createClient { install(ContentNegotiation) { json() } }
    test(client)
}
```

- `testApplication` (Ktor) start de app in het geheugen, zonder echte netwerkpoort.
- Elke test krijgt een eigen, lege database (unieke naam met `System.nanoTime()`), zodat tests elkaar niet beïnvloeden.
- Er wordt **dezelfde** `module(dependencies)` gebruikt als in productie: je test dus echt de plugins, routes en StatusPages.
- `test: suspend (HttpClient) -> Unit`: een parameter die zelf een functie is (**higher-order function**). `suspend` omdat HTTP-aanroepen in Ktor coroutines zijn.

14.3 Handmatig testen

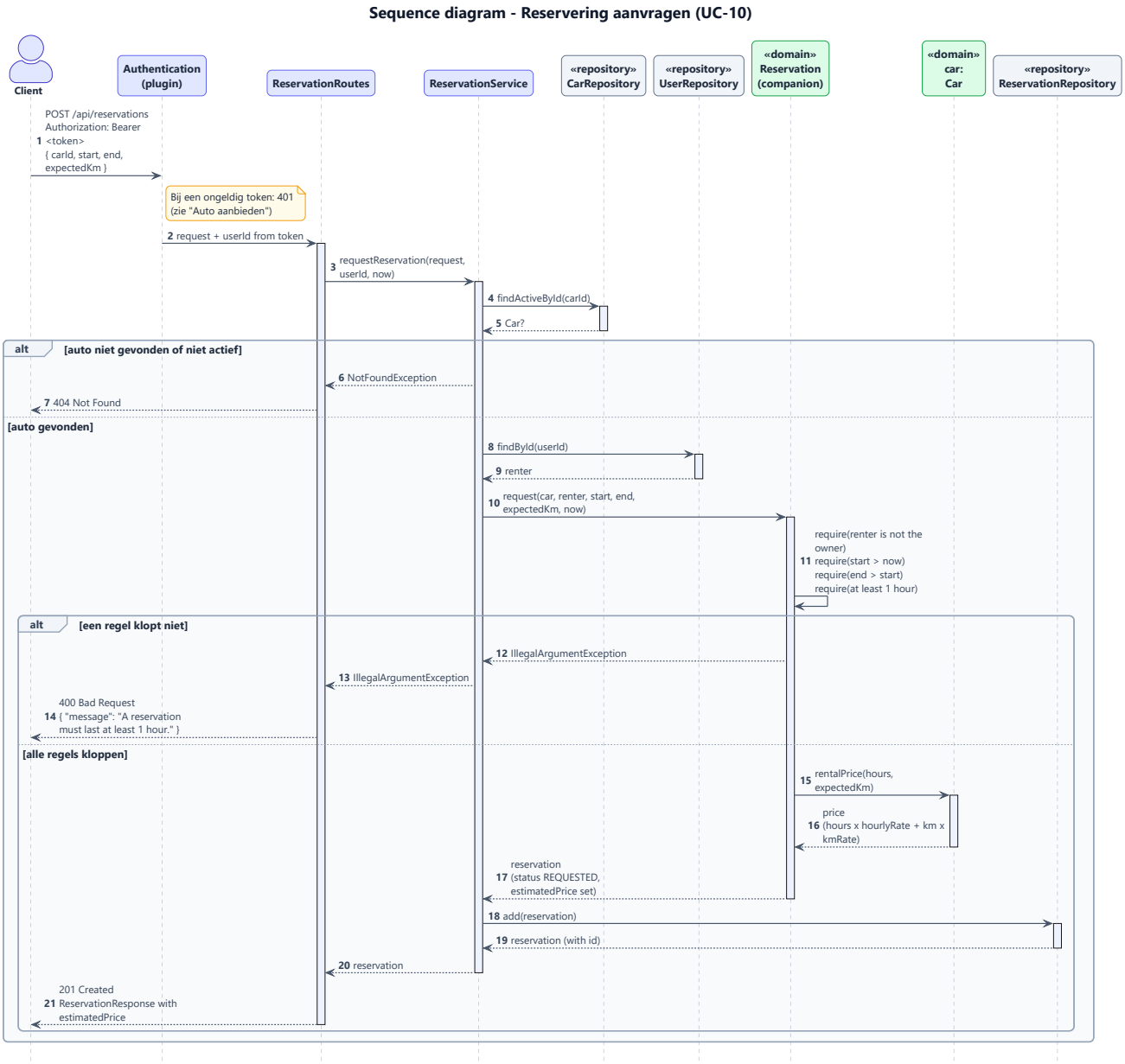
In `server/test/http-requests/` staan `.http`-bestanden (`visitor-flow`, `renter-flow`, `owner-flow`) die je in IntelliJ kunt uitvoeren tegen de draaiende server, per soort gebruiker.

15. Eén request van begin tot eind

Voorbeeld: Eva keurt de reservering van Daan goed. `PATCH /api/reservations/2/status` met body `{ "status": "APPROVED" }` en header `Authorization: Bearer eyJ...`.

1. **Netty** ontvangt het verzoek en geeft het aan Ktor.
2. **Routing** zoekt de passende route: `patch("/reservations/{id}/status")` in `ReservationRoutes.kt`.
3. **Authentication**: de route zit in `authenticate("auth-jwt")`. De verifier controleert handtekening, issuer, audience en verlooptijd. Fout → `challenge` → 401.
4. **Route**: `call.receive<ChangeStatusRequest>()` (ContentNegotiation zet JSON om), `call.intParameter("id")` = 2, `call.userId()` = id van Eva uit het token, `currentTime()`.
5. **Service** `changeStatus`: zoekt de reservering (404 als die niet bestaat), `when` → `approve`: is Eva de eigenaar? (anders 403). In het `synchronized`-blok: overlap met goedgekeurde reserveringen? (409).
6. **Domein** `reservation.approve(now)`: status moet REQUESTED zijn en de start in de toekomst (anders 409 via `check`). Status wordt APPROVED.
7. **Repository** `update`: Exposed schrijft de nieuwe status in de tabel `reservations`.
8. **Mapping** `toResponse()`: omdat de status nu APPROVED is, komt `ownerContact` (naam + e-mail van Eva) in het antwoord. Prijs afgerond op 2 decimalen.
9. **Antwoord**: 200 OK met de `ReservationResponse` als JSON.

Gaat er ergens iets mis, dan gooit die laag een exception, en **StatusPages** maakt er de juiste statuscode + `ApiError` van.



Figuur 6 – Sequentiediagram reservering aanvragen

16. Kotlin-begrippen uitgebreid uitgelegd

Per begrip: **wat** het is, **hoe je het noemt**, **waar** het in de code staat, **waarom** het daar gebruikt is en **wat er zonder** zou gebeuren. Dit zijn precies de vragen die bij een assessment gesteld kunnen worden.

16.1 Variabelen en types

`val` en `var`

Wat: `val` (value) is een waarde die je na het toewijzen niet meer kunt veranderen – *read-only / immutable reference*. `var` (variable) kun je opnieuw een waarde geven – *mutable*.

Waar: bijna alles is `val`: alle velden van `Car`, `Reservation` en de DTO's. `var` alleen bij `User` (naam, e-mail, hash en punten mogen veranderen), bij `isActive` en `status` (met `private set`), en lokaal in `CarService.search` (`var cars = ...` wordt steeds gefilterd) en `Rounding.kt` (`var factor`).

Waarom: hoe minder er kan veranderen, hoe minder er per ongeluk mis kan gaan. Kotlin-advies: "gebruik `val`, tenzij je echt moet veranderen".

Let op: `val` betekent dat de *verwijzing* niet verandert. `val list = mutableListOf()` kun je nog wel vullen (zoals in `Fakes.kt`).

Type inference (type-afleiding)

Wat: je hoeft het type vaak niet te schrijven; de compiler leidt het af. `val car = getActiveCar(id)` is automatisch een `Car`.

Waar wel expliciet: bij publieke functies (`fun findById(id: Int): Car?`) en properties zoals `val verifier: JWTVerifier`, zodat de "buitenkant" van een klasse duidelijk is.

Basistypes

`Int` (heel getal: seats, year, id), `Double` (kommagetal: tarieven, prijzen), `String` (tekst), `Boolean` (`true` / `false`), `ByteArray` (rij bytes: salt en hash), `Unit` (= "niets terug", zoals `void` in Java), `Any` (het moedertype van alles; `Any()` wordt gebruikt als lock-object).

Let op: types beginnen in Kotlin met een hoofdletter (`Int`, niet `int`) – dit staat als fout in de AI-log.

16.2 Null-veiligheid

Nullable types: `T?`

Wat: in Kotlin kan een gewone `String` nooit `null` zijn. Wil je dat wel toestaan, dan schrijf je `String?`. Dit heet **null safety**. De compiler dwingt af dat je de `null`-mogelijkheid afhandelt voordat je de waarde gebruikt.

Waar: `fun findById(id: Int): Car?` (misschien niet gevonden), `fun login(...): String?` (null = inloggen mislukt), `val fuelType: FuelType? = null` in `CarRequest` (alleen bij ICE), `ownerContact: OwnerContact? = null`, `.nullable()`-kolommen.

Waarom: de beroemde `NullPointerException` ("de miljard-dollar-fout") wordt zo al tijdens het compileren voorkomen.

Safe call `?.`, Elvis `?:` en not-null assertion `!!`

`a?.b` **(safe call):** "als `a` niet null is, doe `.b`; anders is het resultaat null".
`parameters["maxHourlyRate"]?.toDouble()`: zonder parameter wordt het gewoon `null` (= niet filteren).

`a ?: b` **(Elvis-operator, naar het kuifje van Elvis Presley als je hem kantelt):** "als `a` null is, neem `b`". Drie vormen in de code:
– standaardwaarde: `System.getenv("JWT_SECRET") ?: DEVELOPMENT_SECRET`
– fout gooien: `userRepository.findById(id) ?: throw NotFoundException("User not found.")`
– stoppen: `findByEmail(request.email) ?: return null`

`a!!` **(not-null assertion, "double bang"):** "ik garandeer dat dit niet null is; is het toch null, crash dan met een exception". Gebruikt in `toCar` (`row[CarTable.fuelType]!!`) en na `add` (`findById(newId.value)!!`). Daar is het veilig, omdat de code zelf net zorgde dat de waarde bestaat. Gebruik het spaarzaam: elke `!!` moet je kunnen verantwoorden.

Smart cast

Wat: na een controle "weet" de compiler meer. Na `if (car == null || !car.isActive) throw ...` is `car` verderop automatisch `Car` (niet meer `Car?`). Na `is IceCar ->` in een `when` is `car` een `IceCar`, dus `car.fuelType` mag.

Waarom: geen handmatige casts (`car as IceCar`) nodig, en dus geen kans op een verkeerde cast.

16.3 Klassen

`class` en de primary constructor

Wat: een klasse is een bouwtekening voor objecten. De haakjes direct na de naam zijn de **primary constructor**. Met `val` / `var` ervoor wordt een parameter meteen een **property** (veld) van het object.

Voorbeeld: `class User(val id: Int, var name: String, ...)` – één regel maakt de klasse, de constructor én de properties. In Java waren dat tientallen regels met getters en setters.

Object maken: `User(1, "Eva", "eva@example.com", "hash")` – geen `new` in Kotlin.

`data class`

Wat: een klasse waarvoor Kotlin automatisch `equals()` (gelijk als alle velden gelijk zijn – *structural equality*), `hashCode()`, `toString()` (bijv. `Address(street=..., city=Breda)`), `copy()` en `componentN()` maakt.

Waar: `Address` en alle DTO's.

Waarom daar: het zijn "zakjes met waarden". `copy()` wordt gebruikt in `Car.toResponse()` en in de tests.

Waarom niét bij User/Car/Reservation: die hebben een eigen identiteit (id) – twee auto's met dezelfde gegevens zijn nog steeds twee auto's. En `toString()` van User zou de hash laten zien. Een gewone klasse vergelijkt op *referential equality* (is het hetzelfde object in het geheugen?).

`init`-blok

Wat: code die automatisch draait direct nadat een object gemaakt is, als onderdeel van de constructor.

Waar: `Car` (alle basiscontroles), `IceCar` / `ElectricCar` / `HydrogenCar` (verbruik > 0), `TokenService` (geheim ≥ 32 tekens).

Waarom: een object controleert zichzelf. Een **invariant** (iets wat altijd waar moet zijn, zoals "uurtarief ≥ 0") kan dan nooit geschonden zijn, waar het object ook vandaan komt (JSON, database, test). Bij een subklasse draait eerst het `init` van `Car`, dan dat van de subklasse.

Properties met `private set`

Wat: `var isActive: Boolean = isActive`
`private set` – iedereen kan `car.isActive` *lezen*, maar alleen code binnen de klasse kan het *veranderen*. Dit heet een property met een **private setter**.

Waarom: encapsulatie (inkapseling): de klasse bewaakt haar eigen toestand. Een reservering kan alleen via `approve()` / `reject()` / `cancel()` van status veranderen, en die functies controleren eerst of dat mag. Zonder `private set` zou een service `reservation.status = APPROVED` kunnen schrijven en alle regels overslaan.

De truc met de constructor: de constructorparameter `isActive: Boolean = true` heeft géén `val`. Het is dus alleen een parameter, waarmee de property in de klasse gevuld wordt. Zo kan de property zelf een private setter krijgen, en kan de repository bij het teruglezen toch de opgeslagen waarde meegeven.

Default arguments en named arguments

Default argument: `bonusPoints: Int = 0`, `isActive: Boolean = true`, `now: Date = Date()`, `fun petrolCar(id: Int = 1, owner: User = OWNER)`. Laat je het argument weg, dan wordt de standaardwaarde gebruikt. Vervangt het maken van meerdere constructors (*overloading*).

Named argument: `IceCar(..., fuelType = FuelType.PETROL, litersPer100Km = 6.0)`, `start(wait = true)`, `rateLimiter(limit = 10, refillPeriod = 1.minutes)`. Je zegt bij welke parameter de waarde hoort. Maakt lange aanroepen leesbaar en voorkomt verwisselingen (6.0 en 0.25 zijn allebei `Double`!).

16.4 Overerving en abstractie

Overerving (inheritance): `class IceCar(...) : Car(...)`

Wat: `IceCar` *is een* `Car`: hij krijgt alle properties en functies van `Car` (de **superklasse** / basisklasse) en voegt eigen dingen toe. `IceCar` is de **subklasse**. De dubbele punt betekent "erft van", en de haakjes erachter roepen de constructor van `Car` aan.

Waarom: alle auto's delen merk, kenteken, tarief, adres, controles en de TCO-formule. Dat staat één keer in `Car` (**DRY**: Don't Repeat Yourself). Alleen het verschil (verbruik en vaste kosten) staat in de subklasse.

Let op: in Kotlin zijn klassen standaard `final` (je kunt er niet van erven). Daarom moet `Car` `open`, `abstract` of `sealed` zijn.

`abstract fun` en `override`

Wat: een abstracte functie heeft alleen een "handtekening" (naam, parameters, returntype), geen inhoud. Elke subklasse **moet** hem invullen met `override fun`. Een klasse met abstracte functies kun je zelf niet maken (`Car(...)` mag niet), alleen haar subklassen.

Waar: `costPerKm()` en `fixedCostsPerYear()`.

Waarom: elke soort auto rekent anders (liters × brandstofprijs, kWh × stroomprijs, kg × waterstofprijs). Vergeet je bij een nieuwe soort de functie, dan compileert de code niet.

Polymorfisme

Wat: letterlijk "veelvormigheid": één aanroep, verschillend gedrag afhankelijk van het echte object. `car.tco()` roept intern `costPerKm()` aan; bij een `IceCar` draait de versie van `IceCar`, bij een `ElectricCar` die van `ElectricCar`. Dit heet ook *dynamic dispatch*.

Waarom: `CarService.getCosts` hoeft niet te weten welke soort auto het is. Geen `if (type == ICE) ... else if ...` in de service.

`sealed class`

Wat: een abstracte klasse waarvan alle subklassen bij het compileren bekend zijn (ze moeten in hetzelfde package en dezelfde module staan). Ook wel "gesloten klassenhiërarchie".

Waar: `sealed class Car` met `IceCar`, `ElectricCar`, `HydrogenCar`.

Waarom: (1) het domein zegt "er zijn precies drie soorten"; (2) een `when (car)` met de drie `is`-takken is **exhaustive** (volledig), dus geen `else` nodig – zie `Car.type()`, `Car.toResponse()` en `writeCar`; (3) voeg je ooit een vierde soort toe, dan wijst de compiler elke `when` aan die je moet aanpassen.

Verschil met `abstract class`: van een abstract class mag iedereen, overal, een subklasse maken. Een `else` in de `when` zou dan verplicht zijn en een nieuwe soort stil verkeerd behandelen.

Verschil met `enum`: een enum-waarde is één vast object zonder eigen gegevens per auto. Een sealed class kan per object eigen data hebben (elk `IceCar`-object heeft zijn eigen `litersPer100Km`).

`interface`

Wat: een contract: een lijst van functies die een klasse moet hebben, zonder (verplichte) inhoud. Een klasse *implementeert* de interface: `class ExposedCarRepository(...) : CarRepository`.

Waar: `UserRepository`, `CarRepository`, `ReservationRepository`.

Waarom: de services hangen af van het contract, niet van de database. Daardoor bestaan er twee implementaties: `Exposed...` (echt) en `Fake...` (tests). Dit heet *programmeren tegen een interface* en is de D uit SOLID (**Dependency Inversion**).

Verschil met abstract class: een interface heeft geen constructor en geen opgeslagen velden; een klasse kan meerdere interfaces implementeren, maar van maar één klasse erven.

Zichtbaarheid: `public`, `private`, `protected`, `internal`

Wat: wie mag iets gebruiken. Standaard is alles `public` (iedereen). `private` in een klasse = alleen die klasse (`private fun makeCar`, `private val algorithm`, `private val carRepository`). `private` op bestandsniveau = alleen dat bestand (`private const val DEVELOPMENT_SECRET`, `private fun atTime`). (`internal` = binnen de module; `protected` = klasse + subklassen; die worden hier niet gebruikt.)

Waarom: hoe kleiner de buitenkant, hoe minder er van buitenaf mis kan gaan, en hoe makkelijker je later iets intern verandert. Het geheime `algorithm` (met het JWT-geheim) is bijvoorbeeld van buiten niet bereikbaar.

16.5 Object, companion object en constanten

`object` (object declaration, singleton)

Wat: `object X { ... }` maakt in één keer een klasse én het enige exemplaar ervan. Dit heet een **singleton**. Je gebruikt het direct via de naam: `CostAssumptions.KM_PER_YEAR`, `DatabaseFactory.connect(url)`. Het wordt gemaakt de eerste keer dat je het gebruikt (lazy).

Waar: `CostAssumptions`, `DatabaseFactory`, `UserTable`, `CarTable`, `ReservationTable`.

Waarom: van kostenaannames en tabellen bestaat er maar één. Het heeft geen zin om `CarTable()` twee keer te maken. Exposed gebruikt tabellen ook standaard als `object`.

Wanneer niet: iets wat per situatie anders is (zoals `TokenService`, die een geheim krijgt) is een gewone klasse, zodat de tests een eigen exemplaar met een test-geheim kunnen maken.

`companion object`

Wat: een `object` *binnen* een klasse. Alles daarin hoort bij de **klasse zelf**, niet bij één object van die klasse. Je roept het aan via de klassenaam: `Car.NEW_ID`, `Reservation.request(...)`, `TokenService.VALID_MINUTES`. Het is de Kotlin-vervanger van `static` in Java/C#. Het heet "companion" (metgezel) omdat het altijd met de klasse meegaat; er is er per klasse maar één.

Waar en waarom:
– `Car.NEW_ID = 0` en `Reservation.NEW_ID = 0`: een constante die over *alle* auto's gaat ("id van een nog niet opgeslagen auto"), niet over één auto. Je hebt hem nodig vóórdat er een auto bestaat: `IceCar(Car.NEW_ID, ...)`. Met een benoemde constante in plaats van een "magisch getal" `0` snapt een lezer wat er bedoeld wordt.
– `Reservation.request(...)`: een **factory-functie**. Je wilt een reservering kunnen maken zonder dat er al een reservering is, dus de functie kan niet op een object staan. Ze hoort wel bij de klasse, omdat ze de regels voor een nieuwe reservering kent. Ze controleert alles (`require`), rekent de prijs uit en roept dan de constructor aan.
– `TokenService.ISSUER`, `AUDIENCE`, `VALID_MINUTES`: vaste waarden van de klasse, voor alle tokens hetzelfde.

Alternatief: een constante op bestandsniveau (zoals `DEMO_PASSWORD` in `DemoData.kt`). Dat kan ook; in de companion staat de constante dichter bij de klasse waar hij bij hoort, en lees je `Car.NEW_ID` (duidelijker dan alleen `NEW_ID`, want Reservation heeft er ook één).

`const val`

Wat: een **compile-time constante**: de waarde staat al vast bij het compileren en wordt overal letterlijk ingevuld. Mag alleen voor basistypes en `String`, en alleen op het hoogste niveau, in een `object` of in een `companion object`.

Verschil met `val`: een `val` kan ook pas tijdens het draaien berekend worden (`val NOW = LocalDateTime(2030, ...)` kan geen `const` zijn, want `LocalDateTime` is geen basistype).

Naam: constanten schrijf je in `SCREAMING_SNAKE_CASE`. `15_000`: underscores in getallen zijn alleen voor de leesbaarheid.

`enum class`

Wat: een type met een vaste lijst benoemde waarden (*enumeratie*). Elke waarde is één object. Je krijgt gratis `.name` ("BEV"), `valueOf("BEV")` en `entries`.

Waarom niet String? Met een String kan "bev", "Bev" of "APROVED" binnenkomen. Met een enum controleert kotlinx.serialization dat de waarde bestaat (anders 400), en de compiler controleert je `when`.

In de database: `enumerationByName` slaat de naam op ("ICE"). Bij `enumeration` (zonder ByName) zou het volgnummer (0, 1, 2) worden opgeslagen, en dan breekt alles als je de volgorde in de enum verandert.

16.6 Functies

Functies, expression body en `Unit`

Block body: `fun register(request: RegisterRequest): User { ...; return ... }`.

Expression body: `fun hours(): Double = hoursBetween(start, end)` – als de functie uit één expressie bestaat, schrijf je `=` in plaats van `{ return ... }`. Ook gebruikt voor `findById(...) = transaction(database) { ... }`: het hele `transaction`-blok is die ene expressie.

Top-level function: een functie buiten een klasse, zoals `currentTime()`, `hoursBetween()`, `main()`, `createDependencies()`. In Kotlin hoeft niet alles in een klasse (in Java wel).

`Unit`: het returntype van een functie die niets teruggeeft (`fun deactivate()`). Je hoeft het niet te schrijven.

Extension functions

Wat: een functie toevoegen aan een bestaand type zonder dat type te veranderen of ervan te erven. Je schrijft het type vóór de naam: `fun Double.rounded(decimals: Int): Double`. Binnen de functie is `this` het object (het getal). Aanroepen: `car.tco().rounded(2)`.

Waar:

– `Application.module()`, `Application.configureStatusPages()` – uitbreiden van Ktor's applicatie;
– `Route.carRoutes()`, `Route.userRoutes()`, `Route.reservationRoutes()` – routes per feature;
– `ApplicationCall.userId()`, `ApplicationCall.intParameter()` – herhaalde code uit de routes;
– `Car.toResponse()`, `User.toResponse()`, `Reservation.toResponse()`, `Car.type()` – omzetten naar DTO;
– `Double.rounded()`.

Waarom bij de mapping? Zo blijft de domeinklasse `Car` vrij van kennis over JSON/DTO's (het domein hoeft `CarResponse` niet te kennen), terwijl je toch `car.toResponse()` kunt schrijven. Elke laag houdt haar eigen verantwoordelijkheid.

Waarom bij Ktor-klassen? Je kunt Ktor's code niet aanpassen, maar zo voelt `call.userId()` alsof het altijd al bij `call` hoorde. Dit is ook hoe Ktor zelf werkt (`install`, `routing`, `get` zijn extension functions).

Lambda's, `it` en trailing lambda

Wat: een lambda is een stukje code dat je als waarde doorgeeft: `{ car -> car.isActive }`. Heeft de lambda één parameter, dan mag je die weglaten en heet hij `it`: `cars.filter { it.isActive }`.

Trailing lambda: is de lambda de laatste parameter, dan mag hij buiten de haakjes: `transaction(database) { ... }`, `require(x) { "melding" }`, `route("/cars") { ... }`. Zijn er verder geen argumenten, dan vervallen de haakjes helemaal: `routing { ... }`, `get { ... }`.

Waarom `require(x) { "melding" }` met een lambda? De tekst wordt alleen gemaakt als de controle faalt (lazy) – een klein beetje efficiënter.

Ongebruikte parameter: `challenge { _, _ -> ... }` – een `_` zegt "deze parameter heb ik niet nodig".

Lambda met ontvanger en DSL

Wat: een lambda waarin `this` een bepaald object is. In `install(ContentNegotiation) { json(...) }` is `this` de configuratie van ContentNegotiation, daarom kun je direct `json(...)` aanroepen. In `it[UserTable.name] = name` (bij `insertAndGetId { }`) is `it` de nieuwe rij.

DSL (Domain Specific Language): code die leest als een eigen taaltje voor één doel. De routing (`route("/cars") { get { } post { } }`), de Exposed-queries (`selectAll().where { CarTable.id eq id }`) en de `Json { prettyPrint = true }`-configuratie zijn DSL's die met lambda's met ontvanger gebouwd zijn.

Higher-order functions en collectie-functies

Wat: een functie die een functie als parameter krijgt of teruggeeft. Het type van een functieparameter schrijf je als `(HttpClient) -> Unit`: "krijgt een HttpClient, geeft niets terug". Zie `apiTest(test: suspend (HttpClient) -> Unit)`.

Collectie-functies in de code:

– `map { }`: elk element omzetten → nieuwe lijst (`cars.map { it.toResponse() }`);
– `filter { }`: alleen elementen waarvoor de lambda `true` geeft;
– `any { }`: is er minstens één die voldoet? (overlap, toekomstige reserveringen);
– `find { }`: eerste die voldoet, of null (in de fakes);
– `singleOrNull()`: precies één, of null;
– `removeIf { }`, `toList()`, `count()`, `split(":")`.

Waarom: korter en duidelijker dan zelf een `for`-lus met een hulplijst schrijven; de bedoeling staat in de naam.

Scope function `let`

Wat: `x?.let { ... }` voert het blok alleen uit als `x` niet null is; in het blok is `it` de waarde, en het resultaat van het blok is het resultaat van `let`.

Waar: `parameters["type"]?.let { CarType.valueOf(it.uppercase()) }` – alleen omzetten naar een enum als de parameter er is.

`when` als expressie

Wat: de Kotlin-versie van `switch`, maar krachtiger: werkt met waarden (`when (fuelType) { FuelType.PETROL -> ... }`), types (`is IceCar ->`) en geeft een waarde terug (`return when (...) { ... }`).

Exhaustive: bij een enum of sealed class moet je alle gevallen afhandelen als je het resultaat gebruikt; dan is `else` overbodig. Zie `fuelPrice`, `Car.type()`, `makeCar`, `changeStatus`.

String templates en `Regex`

String template: `"jdbc:h2:mem:$name;DB_CLOSE_DELAY=-1"` en `"/api/cars/${car.id}"` – met `$naam` of `${expressie}` zet je een waarde in een tekst.

Regex: `email.matches(Regex("^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$"))` – een patroon: tekens vóór de @, een domein, een punt, minstens 2 letters. `\\.` = een echte punt (de backslash moet dubbel in een Kotlin-string).

16.7 Fouten (exceptions)

Exceptions, `throw`, `try/catch`

Wat: een exception is een object dat zegt "hier ging iets mis". `throw` gooit hem; de normale uitvoering stopt en de exception "bubbelt" omhoog door de aanroepende functies, tot iemand hem vangt met `try { } catch (e: X) { }`. Hier vangt de StatusPages-plugin ze voor alle routes tegelijk.

`throw` **is een expressie** (van type `Nothing`), daarom mag het rechts van `?:`: `request.fuelType ?: throw IllegalArgumentException(...)`.

`try` **is ook een expressie:** in `TokenService.isValid`: `return try { verifier.verify(token); true } catch (e: JWTVerificationException) { false }`.

Eigen exceptions: `class ForbiddenException(message: String) : Exception(message)` – een eigen soort, zodat StatusPages er een eigen statuscode (403) aan kan koppelen.

`require`, `check` (en `error`)

`require(voorwaarde) { melding }`: "de invoer moet hieraan voldoen". Gooit `IllegalArgumentException` → 400. Voor **precondities** op argumenten.

`check(voorwaarde) { melding }`: "het object moet in deze toestand zijn". Gooit `IllegalStateException` → 409. Voor toestandsregels (status, tijd).

Waarom niet `if (...) throw ...`**?** Dat mag ook (en wordt gedaan voor de eigen exceptions), maar `require` / `check` zijn korter en zeggen meteen wat voor soort controle het is.

16.8 Overige

Generics

Wat: code die met verschillende types werkt, waarbij je het type als parameter meegeeft tussen `< >`. `List<Car>` = een lijst van auto's; `call.receive<CarRequest>()` = lees de body als CarRequest; `principal<JWTPrincipal>()`; `enumerationByName<CarType>(...)`; `assertIs<IceCar>(ice)`; `UpdateBuilder<*>` (`*` = "maakt niet uit welk type").

Waarom: typeveiligheid: de compiler weet dat in een `List<Car>` alleen auto's zitten.

Annotations

Wat: labels met een `@` die extra informatie geven aan de compiler of een library. `@Serializable` → maak JSON-code; `@Test` → dit is een testfunctie.

Infix-functies

Wat: een functie met één parameter die je zonder punt en haakjes mag schrijven: `CarTable.id eq id` betekent `CarTable.id.eq(id)`. Ook `innerJoin` en `and` in Exposed. Maakt queries leesbaar als SQL.

Coroutines en `suspend`

Wat: Ktor is gebouwd op **coroutines**: "lichte threads" waarmee de server duizenden verzoeken tegelijk kan afhandelen zonder voor elk een zware thread te gebruiken. Een `suspend fun` kan pauzeren (bijvoorbeeld wachten op het netwerk) zonder de thread te blokkeren.

Waar zie je het: de blokken in `get { }`, `post { }` en `testApplication { }` zijn suspend-lambda's; daarom mag je daarin `call.receive()`, `call.respond()` en `client.post()` aanroepen. In de eigen code komt `suspend` alleen voor in `apiTest`.

Threads, race condition en `synchronized`

Thread: een "draad" van uitvoering. Een server verwerkt verzoeken op meerdere threads tegelijk.

Race condition: een fout die ontstaat als twee threads tegelijk dezelfde data lezen en schrijven, en de uitkomst afhangt van wie "wint". Voorbeeld: twee eigenaars-verzoeken keuren tegelijk twee overlappende reserveringen goed.

`synchronized(lock) { }`**:** maar één thread tegelijk mag het blok in; de anderen wachten. Het lock-object (`Any()`) is de "sleutel van de wc-deur". Het blok is klein gehouden (alleen controleren + opslaan), zodat andere verzoeken er nauwelijks op hoeven te wachten.

17. Ontwerppatronen en hoe je ze noemt

Een **ontwerppatroon** (design pattern) is een bekende, herbruikbare oplossing voor een veelvoorkomend probleem. Als je de naam kent, kun je je keuzes kort uitleggen bij het assessment.

Naam	Wat het is	Waar in de code	Waarom
Gelaagde architectuur (layered architecture)	De code is verdeeld in lagen met elk één taak; een laag gebruikt alleen de laag eronder.	Routes → Service → Domein/Repository	Overzicht, testbaarheid, wijzigingen blijven lokaal.
Package by feature	Indeling in mappen per onderwerp in plaats van per laag.	`users/`, `cars/`, `reservations/`	Alles van één feature bij elkaar.
Repository pattern	Een klasse die doet alsof het een verzameling objecten is (zoeken, toevoegen) en de database verbergt.	`CarRepository` + `ExposedCarRepository`	Service kent geen SQL; database is vervangbaar; fakes in tests.
Service layer	Eén plek voor de use cases van de applicatie.	`CarService`, ...	Regels (eigenaarschap, conflicten) niet in de routes.
DTO (Data Transfer Object)	Aparte klassen voor wat over het netwerk gaat.	`core/*Dtos.kt`	Geen hash lekken; API-vorm los van het domein.
Mapper	Omzetten tussen twee vormen.	`toResponse()`, `toCar()`, `writeCar()`	Omzetting op één plek.
Dependency injection (constructor injection, "poor man's DI")	Een klasse krijgt haar afhankelijkheden via de constructor in plaats van ze zelf te maken.	`Dependencies.kt`, alle services	Losse koppeling; tests kunnen andere onderdelen meegeven. Zonder framework: alles is zichtbaar.
Composition root	De ene plek waar alles aan elkaar gekoppeld wordt.	`createDependencies()`	Nergens anders worden services "zomaar" gemaakt.
Singleton	Precies één exemplaar.	`object CostAssumptions`, tabellen	Er bestaat er logisch maar één.
Factory method	Een functie die een object maakt (en controleert) in plaats van direct de constructor.	`Reservation.request()`, `CarService.makeCar()`	Regels en berekeningen bij het maken op één plek; de juiste subklasse kiezen.
Rich domain model	Domeinklassen bevatten zelf hun regels en gedrag (niet alleen data).	`Car.tco()`, `Reservation.approve()`, `init`-controles	Een object kan nooit in een ongeldige toestand komen.

Value object	Object zonder identiteit, gelijk als de waarden gelijk zijn.	`Address`	Eenvoudig te vergelijken en te kopiëren.
Entity	Object met eigen identiteit (id).	`User`, `Car`, `Reservation`	Blijft "hetzelfde" ook als gegevens veranderen.
Toestandsmachine (state machine)	Een vast aantal toestanden met toegestane overgangen.	`ReservationStatus` + `approve/reject/cancel`	Onmogelijke overgangen (afgewezen → goedgekeurd) worden geweigerd.
Template method	De superklasse heeft het algoritme, subklassen vullen stappen in.	`Car.tco()` met `costPerKm()` / `fixedCostsPerYear()`	Eén formule, drie invullingen.
Single table inheritance	Een klassenhiërarchie in één tabel met een type-kolom.	`CarTable`	Eenvoudig; geen joins.
Soft delete	Niet echt verwijderen, maar markeren als inactief.	`Car.deactivate()`	Historie en verwijzingen blijven kloppen.
Test double / fake	Vervanger van een echt onderdeel in tests.	`Fakes.kt`	Snelle unit tests zonder database.
Fixed clock (tijd injecteren)	De huidige tijd als parameter in plaats van zelf opvragen.	`now`-parameters, `NOW` in tests	Tests zijn herhaalbaar (deterministisch).
Centrale foutafhandeling	Eén plek die exceptions vertaalt naar antwoorden.	`StatusPages.kt`	Geen try/catch in elke route; altijd dezelfde foutvorm.

17.1 SOLID in deze code (kort)

Letter	Principe	Voorbeeld
S	Single Responsibility – één reden om te veranderen	`PasswordHasher` doet alleen hashen; `TokenService` alleen tokens; routes alleen HTTP.
O	Open/Closed – uitbreiden zonder bestaande code te breken	Nieuwe soort auto = nieuwe subklasse; de compiler wijst de `when`'s aan.
L	Liskov Substitution – een subklasse moet overal kunnen waar de superklasse verwacht wordt	Elke `IceCar` / `ElectricCar` / `HydrogenCar` werkt in `getCosts`, `search`, `rentalPrice`.
I	Interface Segregation – kleine, gerichte interfaces	Drie aparte repository-interfaces in plaats van één grote.
D	Dependency Inversion – afhangen van abstracties	Services krijgen `CarRepository` (interface), niet `ExposedCarRepository`.

18. Spiekbrief Kotlin-concepten

Concept	Voorbeeld uit de code	In één zin
`val` / `var`	`val id`, `var name`	Niet / wel opnieuw toe te wijzen.
Nullable type `T?`	`fun findById(id: Int): Car?`	De waarde mag `null` zijn; de compiler dwingt je dat af te handelen.
Safe call `?.`	`parameters["type"]?.let { ... }`	Alleen uitvoeren als niet null.
Elvis `?:`	`findById(id) ?: throw NotFoundException(...)`	Als links null is, gebruik rechts.
Not-null assertion `!!`	`row[CarTable.fuelType]!!`	"Ik weet zeker dat het niet null is" – anders crash.
`class`	`User`, `Reservation`	Object met identiteit en gedrag.
`data class`	`Address`, alle DTO's	Automatisch equals/hashCode/toString/copy; voor waarden.
`sealed class`	`Car`	Vaste, bekende set subklassen; `when` zonder else.
`abstract` / `override`	`costPerKm()`	Subklasse moet de functie invullen.
`interface`	`CarRepository`	Contract zonder implementatie; meerdere versies mogelijk.
`enum class`	`CarType`	Vaste lijst van waarden.
`object`	`CostAssumptions`, `UserTable`	Singleton: precies één exemplaar.
`companion object`	`Car.NEW_ID`, `Reservation.request()`	Hoort bij de klasse, niet bij een object ("static").
`const val`	`KM_PER_YEAR = 15_000`	Constante die bij het compileren al bekend is.
`init`-blok	In `Car`	Code die draait bij het maken van het object.
`require` / `check`	Car, Reservation	Invoer (→ IllegalArgument, 400) / toestand (→ IllegalState, 409).
`private set`	`isActive`, `status`	Lezen mag iedereen, wijzigen alleen de klasse zelf.
Default / named arguments	`isActive: Boolean = true`, `wait = true`	Parameter met standaardwaarde / argument bij naam.
Extension function	`Car.toResponse()`, `Double.rounded()`	Functie toevoegen aan een bestaand type.
Lambda / `it`	`cars.filter { it.isActive }`	Een stukje code als waarde; `it` = enige parameter.
Higher-order function	`apiTest(test: suspend (HttpClient) -> Unit)`	Functie die een functie als parameter krijgt.
`when`	`when (car) { is IceCar -> ... }`	Uitgebreide switch; ook een expressie.

Smart cast	Na `is IceCar ->`	Compiler behandelt de variabele automatisch als dat type.
String template	`"'$name' must be a number."`	Waarde in tekst invoegen.
Expression body	`fun hours(): Double = hoursBetween(start, end)`	Korte functie met `=`.
Infix-functie	`CarTable.id eq id`, `innerJoin`	Functie aanroepen zonder punt en haakjes.
`throw` als expressie	`request.fuelType ?: throw ...`	Mag rechts van `?:`.
Generics	`call.receive<CarRequest>()`, `List<Car>`	Type als parameter.
Annotation	`@Serializable`, `@Test`	Extra informatie voor compiler/library.
`synchronized`	`synchronized(approveLock) { ... }`	Maar één thread tegelijk in dit blok.

19. Oefenvragen voor het assessment

Probeer eerst zelf te antwoorden, lees dan het antwoord.

1. **Waarom is `Car` een `sealed class` en geen gewone `open class` of interface?**
   Er zijn precies drie soorten auto's. Met sealed weet de compiler dat, dus een `when (car)` heeft geen `else` nodig en geeft een fout als er later een soort bijkomt die je vergeet. Een abstract class (wat sealed ook is) kan gedeelde velden, controles en `tco()` bevatten; een interface niet.

2. **Wat is het verschil tussen `require` en `check`, en welke statuscode volgt eruit?**
   `require` controleert argumenten/invoer en gooit `IllegalArgumentException` → 400. `check` controleert de toestand van het object en gooit `IllegalStateException` → 409. Voorbeeld: negatieve prijs = require; een al afgewezen reservering goedkeuren = check.

3. **Waarom is `User` geen data class, maar `Address` wel?**
   Een gebruiker heeft een eigen identiteit (id) en twee gebruikers met dezelfde naam zijn niet dezelfde persoon. Ook zou de `toString()` van een data class de wachtwoord-hash tonen. Een adres is een waarde: gelijke velden = gelijk adres.

4. **Hoe zorg je dat niemand de auto van een ander kan wijzigen?**
   De route geeft het id uit het JWT-token mee (`call.userId()`). De service roept `getOwnCar(id, userId)` aan, die `ForbiddenException` gooit (403) als `car.owner.id != userId`. De eigenaar komt nooit uit de JSON.

5. **Wat is het verschil tussen 401 en 403?**
   401: we weten niet wie je bent (geen, verlopen of ongeldig token). 403: we weten wie je bent, maar je mag dit niet.

6. **Waarom staat de overlapcontrole in een `synchronized`-blok?**
   Ktor verwerkt verzoeken tegelijk. Zonder lock kunnen twee goedkeuringen tegelijk "geen overlap" vinden en allebei opslaan (race condition). Het lock zorgt dat controleren en opslaan één ondeelbare stap zijn.

7. **Waarom wordt een auto niet echt verwijderd?**
   Soft delete: `deactivate()` zet `isActive = false`. Oude reserveringen verwijzen nog naar de auto (foreign key) en blijven kloppen. Verwijderen kan niet als er nog open reserveringen in de toekomst zijn (409).

8. **Waarom krijgen functies een `now`-parameter in plaats van zelf de tijd op te vragen?**
   Zodat tests een vaste tijd kunnen meegeven en altijd hetzelfde resultaat geven. Alleen de route vraagt `currentTime()` op.

9. **Leg uit hoe wachtwoorden worden opgeslagen.**
   Met Argon2id (BouncyCastle, aanbevolen door OWASP), met een willekeurige salt van 16 bytes per wachtwoord. Opgeslagen als "salt:hash" in Base64. Bij inloggen wordt met dezelfde salt opnieuw gehasht en in constante tijd vergeleken.

10. **Wat staat er in het JWT-token en hoe lang is het geldig?**
    Issuer, audience, subject (gebruikers-id), issued-at en expires-at; ondertekend met HMAC256. 15 minuten geldig. De inhoud is leesbaar maar niet te vervalsen zonder het geheim.

11. Waarom hebben de services een interface als repository-parameter?

Zodat je in unit tests een fake (lijst in het geheugen) kunt meegeven en in productie de Exposed-versie. De service hoeft niet te weten welke het is.

12. Hoe worden drie soorten auto's in één tabel opgeslagen en weer teruggelezen?

Single table inheritance: kolom `type` (ICE/BEV/FCEV) plus nullable kolommen voor het verbruik. `writeCar` zet eerst alle verbruikskolommen op null en vult dan die van de soort; `toCar` kijkt naar `type` en maakt de juiste subklasse.

13. Wat gebeurt er als iemand `GET /api/cars/abc` doet?

`intParameter("id")`: `"abc".toIntOrNull()` is null, dus `BadRequestException` → StatusPages → 400 met `{ "message": "'id' must be a number." }`.

14. Wat betekent `fun Route.carRoutes(...)`?

Een extension function op Ktor's `Route`. Je kunt hem aanroepen binnen `route("/api") { carRoutes(...) }`, alsof het een functie van Route zelf is. Zo heeft elke feature zijn eigen routesbestand.

15. Reken de TCO van de Tesla Model 3 uit (€ 35.000, 17 kWh/100 km).

Afschrijving 35.000 × 0,15 = 5250. Vaste kosten 1100 + 710 + 385 = 2195. Kosten per km 17 × 0,34 / 100 = 0,0578; × 15.000 = 867. TCO = 5250 + 2195 + 867 = **€ 8312**.

16. Waarom wordt bij inloggen hetzelfde antwoord gegeven bij een onbekend e-mailadres en een fout wachtwoord?

Anders kan een aanvaller uitzoeken welke e-mailadressen een account hebben (user enumeration). Samen met rate limiting (10/min) maakt dat raden moeilijk.

17. Wanneer ziet een huurder de contactgegevens van de eigenaar?

Alleen als de reservering APPROVED is; dat regelt `Reservation.toResponse()`.

18. Waarom maakt `updateCar` een nieuw `Car`-object?

De velden zijn `val`. Door een nieuw object te maken worden alle controles in de `init`-blokken opnieuw uitgevoerd, zodat een wijziging nooit een ongeldige auto oplevert.

20. Aandachtspunten in de huidige code

Dingen die de AI opviel bij het lezen. Het zijn geen grote fouten, maar goed om te weten (en mogelijk vragen van een docent):

- **`core/src/GreetingUtil.kt`** is nog een overblijfsel van het projectsjabloon (package `org.example.project`, functie `sayHello`). Wordt nergens gebruikt; kan weg.
- **Java-libraries**: `PasswordHasher` gebruikt `java.security.SecureRandom` / `MessageDigest` en `TokenService` gebruikt `java.util.Date`. Dat is in lijn met wat de libraries (BouncyCastle, java-jwt) nodig hebben, maar wijkt af van "Kotlin-libraries waar mogelijk". Zorg dat je kunt uitleggen waarom.
- **Zoekfilter**: `GET /api/cars?type=XYZ` geeft via `CarType.valueOf` een `IllegalArgumentException` → 400, en `maxHourlyRate=abc` geeft via `toDouble()` een `NumberFormatException`. Die laatste is een subklasse van `IllegalArgumentException`, dus ook 400. Goed om te weten waarom dat werkt.
- **`DemoData.plusHours`** telt uren op bij het uur van de dag. Met 9 + 4 en 10 + 3 gaat dat goed, maar boven 23 uur zou `LocalTime` een fout geven. Alleen relevant als je de demodata verandert.
- **`synchronized`** beschermt alleen binnen één serverproces. Met meerdere servers zou je een databaseoplossing nodig hebben (buiten de scope van dit project).
- **Bonuspunten** staan in `User` en de tabel, maar worden nog nergens opgehoogd.

Tot slot
Kun je bij elk bestand in één zin zeggen wat het doet, en bij elke regel waarom hij er staat? Dan ben je klaar voor het assessment. Gebruik de oefenvragen en leg de code eens hardop uit aan iemand anders.